

GNN TEACHING WORKSPACE

Graph Neural Networks

A complete 23-lesson course — from graph fundamentals through message passing, the major architectures, theory, and temporal GNNs for fraud & AML detection.

Compiled from the `/teach` workspace lessons.

Table of Contents

1. Lesson 1: Why Graphs? What Graphs Encode
2. Lesson 2: Graph Representations in Practice
3. Lesson 3: The Three Prediction Tasks
4. Lesson 4: Message Passing: How Nodes Talk to Neighbors
5. Lesson 5: Stacking Layers and the Receptive Field
6. Lesson 6: Over-smoothing and the Depth Problem
7. Lesson 7: GCN: Spectral Roots, Spatial Practice
8. Lesson 8: GraphSAGE: Inductive Learning and Sampling
9. Lesson 9: GAT: Attention on Graphs
10. Lesson 10: GIN: Maximising Expressiveness
11. Lesson 11: Architecture Comparison
12. Lesson 12: Readout and Pooling for Graph-Level Tasks
13. Lesson 13: Loss Functions and Training Strategies
14. Lesson 14: Node Embeddings Before GNNs
15. Lesson 15: The Weisfeiler-Leman Hierarchy
16. Lesson 16: Geometric Deep Learning: The Symmetry Perspective
17. Lesson 17: Graph Transformers
18. Lesson 18: Heterogeneous Graphs
19. Lesson 19: Knowledge Graphs and Embedding
20. Lesson 20: Dynamic and Temporal GNNs
21. Lesson 21: GNNs for Fraud and AML Detection
22. Lesson 22: Deep Generative Models for Graphs
23. Lesson 23: LLMs + GNNs

Why Graphs? What Graphs Encode

You already know how CNNs exploit the grid structure of images and RNNs exploit the sequential structure of text. GNNs are the answer to a natural question: *what if the data has neither grid nor sequence structure, but relational structure?*

A graph is a triple

A graph $\mathbf{G} = (\mathbf{V}, \mathbf{E}, \mathbf{u})$ consists of:

- **Nodes** (vertices) $\mathbf{V}$ — the entities
- **Edges** $\mathbf{E} \subseteq \mathbf{V} \times \mathbf{V}$ — the relationships between entities
- **Global attributes** $\mathbf{u}$ — properties of the entire graph

Each component carries a **feature vector**:

- Node features: a matrix $\mathbf{X} \in \mathbb{R}^{|\mathbf{V}| \times \mathbf{d}_v}$
- Edge features: a matrix $\mathbf{E} \in \mathbb{R}^{|\mathbf{E}| \times \mathbf{d}_e}$
- Global features: a vector $\mathbf{u} \in \mathbb{R}^{\mathbf{d}_u}$

Key idea: A graph encodes *what things are* (node/edge features) separately from *how they relate* (connectivity structure). This separation is what makes graphs powerful — the same GNN architecture works whether a molecule has 5 atoms or 50.

Data you already know, reframed as graphs

The insight is that grids and sequences are *special cases* of graphs:

Data	Nodes	Edges	Structure
Image	Pixels	Adjacency (4 or 8 neighbors)	Regular grid
Text	Tokens	Sequential links	Chain
Molecule	Atoms	Bonds	Irregular
Social network	People	Friendships	Irregular, scale-free
Citation network	Papers	Citations	Directed, irregular

Analogy: A CNN is a GNN on a grid graph with weight-sharing across the spatial kernel. An RNN is a GNN on a chain graph with weight-sharing across time steps. GNNs generalise both to *arbitrary* topologies.

Why not just flatten it?

You might ask: why not represent the adjacency matrix as a flat vector and feed it into an MLP? Three reasons:

1. **Variable size.** Graphs vary in node count. An MLP needs a fixed input dimension. You'd need padding, which wastes parameters and doesn't generalise.
2. **Permutation sensitivity.** The same graph can be represented by any reordering of its adjacency matrix. An MLP would treat each reordering as a different input. A GNN is *permutation invariant* (or equivariant) by design.
3. **No structural inductive bias.** An MLP doesn't know that node 3 and node 7 are neighbors. A GNN's operations are *defined by* the connectivity, so it naturally respects the structure.

Key idea: The defining property of a GNN is **permutation equivariance** — reordering the nodes produces the same reordering in the output, rather than

a completely different result. This is the graph analogue of translation equivariance in CNNs.

Three levels of prediction

GNNs can make predictions at three levels, each requiring a different output head:

1. **Node-level:** classify each node (e.g., predict the role of a protein in a PPI network)
2. **Edge-level:** predict a property per edge (e.g., does a link exist? what type of bond?)
3. **Graph-level:** predict a property of the whole graph (e.g., is this molecule toxic?)

The architecture stays mostly the same — only the final pooling/readout step changes. We'll cover this in detail in a later lesson.

Check your understanding

1. A social network where users have profile features and friendships have a "since" date. What carries the "since" date?

Node feature vector

Edge feature vector

Global feature vector

Adjacency matrix entry

The "since" date is a property of the *relationship* between two users, so it belongs to the edge feature vector. The adjacency matrix only stores connectivity (0 or 1), not rich

attributes.

2. Why is permutation equivariance important for a GNN?

It reduces the parameter count of the model

It allows the model to handle directed edges

It ensures the output is independent of arbitrary node ordering

It enables the model to process variable-length sequences

Graphs have no canonical node ordering. Two isomorphic graphs should produce the same predictions regardless of how the nodes are indexed. Permutation equivariance guarantees this — reorder the input, and the output reorders consistently.

3. Predicting whether a molecule is soluble in water is an example of which prediction level?

Node-level prediction task

Edge-level prediction task

Graph-level prediction task

Subgraph-level prediction

Solubility is a property of the entire molecule, not of any single atom or bond. This requires graph-level prediction: pool all node representations into a single graph representation, then classify.

Primary source

Read: "A Gentle Introduction to Graph Neural Networks" (<https://distill.pub/2021/gnn-intro/>) by Sanchez-Lengeling et al. (Distill, 2021). Focus on **Sections 1–3** (What is a graph? / Data as graphs / GNN predictions). The interactive diagrams make the node/edge/global distinction visceral.

Questions about anything in this lesson? Ask your teacher — run `/teach` and ask about whatever is unclear. Next up: **Lesson 2 — Graph Representations in Practice**, where we'll cover how graphs actually get stored as tensors for a neural network.

← Start

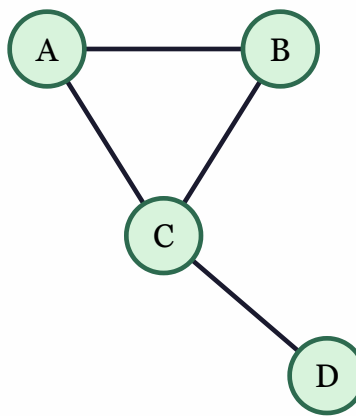
Lesson 2: Graph Representations in Practice →

Graph Representations in Practice

Lesson 1 defined a graph abstractly as $G = (\mathbf{V}, \mathbf{E}, \mathbf{u})$ with node, edge, and global feature matrices. Before we can write a single line of message-passing code, we need to answer a more mundane question: what tensors actually go into the model? This lesson is about the plumbing.

The example graph

We'll use one small graph throughout. Four nodes, four undirected edges:



Nodes 0=A, 1=B, 2=C, 3=D. Edges: A-B, A-C, B-C, C-D.

The node ordering (A=0, B=1, C=2, D=3) is arbitrary — recall from Lesson 1 that a GNN must be permutation equivariant to this choice — but every representation below has to agree on *some* fixed ordering to stay consistent.

Representation 1: the dense adjacency matrix

The most direct encoding of connectivity is the matrix $\mathbf{A} \in \{0,1\}^{|\mathbf{V}| \times |\mathbf{V}|}$ from the glossary, with $A_{ij} = 1$ if an edge connects i and j :

	A	B	C	D
A	0	1	1	0
B	1	0	1	0
C	1	1	0	1
D	0	0	1	0

Clean and intuitive — and this is exactly the object the original GCN paper (Kipf & Welling) writes its propagation rule in terms of. But it costs $O(|V|^2)$ memory regardless of how few edges exist. Real-world graphs are almost always **sparse**: a social network with a million users might have a few hundred edges per node, not a million. Storing 10^{12} mostly-zero entries to represent 10^8 edges is wasteful in both memory and compute.

Representation 2: the edge list (COO format)

Instead of storing the whole matrix, store only the coordinates of the nonzero entries — the pairs of connected nodes. This is called **COO (coordinate) format**, and it's what PyTorch Geometric calls `edge_index`: a tensor of shape **[2, |E|]**, where column i gives the (source, target) pair for the i -th edge.

Key idea: Because message passing is defined edge-by-edge (gather from neighbors, aggregate, update — more on this in Lesson 4), a GNN never actually needs the full $|V| \times |V|$ matrix. It only needs to iterate over existing edges. The edge list is the natural, sparse-friendly format for that computation, which is why libraries like PyTorch Geometric and DGL use it as their default.

For our graph, treating each undirected edge as *two* directed entries (so information flows both ways during message passing):

```
edge_index = [[0, 1, 0, 2, 1, 2, 2, 3],    # source
              [1, 0, 2, 0, 2, 1, 3, 2]]    # target
# shape: [2, 8] → 4 undirected edges stored as 8 directed entries
```

Analogy: This is exactly the sparse-matrix trick you've likely seen elsewhere — SciPy's `coo_matrix`, or how a sparse attention mask gets stored. A GNN's edge list is a sparse adjacency matrix in disguise, chosen because real graphs, like real sparse matrices, are mostly zeros.

The node feature matrix, and why row order matters

Alongside `edge_index` sits the node feature matrix $\mathbf{X} \in \mathbb{R}^{|V| \times d}$ from Lesson 1. Row i of $\mathbf{X}$ must correspond to node i as used in `edge_index` — there is no separate "node ID" field. If A/B/C/D have 3-dimensional features:

```
X = [[x_A0, x_A1, x_A2],    # row 0 → node A
     [x_B0, x_B1, x_B2],    # row 1 → node B
     [x_C0, x_C1, x_C2],    # row 2 → node C
     [x_D0, x_D1, x_D2]]    # row 3 → node D
# shape: [4, 3]
```

This is a common source of silent bugs when building a graph dataset by hand: if the row order in $\mathbf{X}$ doesn't match the index scheme in `edge_index`, the model trains without error but learns garbage connectivity.

Self-loops

You'll frequently see edge lists augmented with a "self-loop" for every node — an edge (i, i) . This isn't part of the original graph; it's added so that when a node aggregates messages from its neighbors, it also includes its own current features in

that aggregation. We'll see exactly why this matters when we derive the GCN update rule in Lesson 7 — for now, just recognize `edge_index` entries where source equals target as this deliberate addition, not a data error.

Batching multiple graphs

Training rarely happens on one graph. For graph-level tasks (Lesson 3), you have a whole dataset of small graphs — thousands of molecules, say — and want to process a batch of them together, the way you'd batch images. But graphs in a batch have different numbers of nodes, so you can't just stack them into a new tensor dimension the way you stack fixed-size images.

The standard trick: concatenate all graphs in the batch into *one big graph* with no edges between the original graphs (a block-diagonal adjacency structure), and keep a separate **batch vector** that records which original graph each node belongs to.

Key idea: Batching in a GNN doesn't add a batch dimension — it disjoint-ions the graphs into one larger sparse graph, plus an index vector. Message passing naturally respects this because it only ever looks at `edge_index`, and there are no edges connecting nodes from different original graphs, so no cross-contamination happens. The readout step (Lesson 12) then uses the batch vector to know which nodes to pool together.

Check your understanding

1. Which format avoids wasting memory on the many zero entries of a large, sparse graph?

Dense adjacency matrix

Sparse edge list

Full feature matrix

One-hot node vector

A dense adjacency matrix always costs $O(|V|^2)$ regardless of edge count. An edge list (COO format / `edge_index`) stores only the pairs that actually have an edge, which scales with $|E|$ instead — the right choice when $|E| \ll |V|^2$, as in nearly all real graphs.

2. In an `edge_index` tensor of shape $[2, |E|]$, what does column i represent?

The i -th node feature

The i -th edge's endpoints

The total node count

The graph's global attribute

Each column is one directed edge, given as a (source, target) pair of node indices. Node features live in a separate matrix X , indexed by the same node IDs that appear inside `edge_index`.

3. When batching several graphs together for training, how are they kept from exchanging messages with each other?

It pads every graph identically

It merges all node features

It keeps graphs fully disconnected

It averages every edge weight

Batching concatenates the graphs into one larger graph with no edges added between them — a block-diagonal structure. Since message passing only follows existing edges, nodes from different original graphs simply never appear as each other's neighbors.

Primary source

Read: "Graph Representation Learning" (https://www.cs.mcgill.ca/~wlh/grl_book/) by William L. Hamilton, **Chapters 1–2**, for the formal treatment of graph types, adjacency structure, and the properties (degree distribution, connectivity, sparsity) that make real-world graphs the way they are. For a hands-on companion showing this exact `edge_index` / batching format in code, see PyTorch Geometric's "Introduction by Example" (https://pytorch-geometric.readthedocs.io/en/latest/get_started/introduction.html) tutorial.

Questions about anything in this lesson? Ask your teacher — run `/teach` and ask about whatever is unclear. Next up: **Lesson 3 — The Three Prediction Tasks**, where we'll see how node-, edge-, and graph-level tasks each shape the final layers of a GNN.

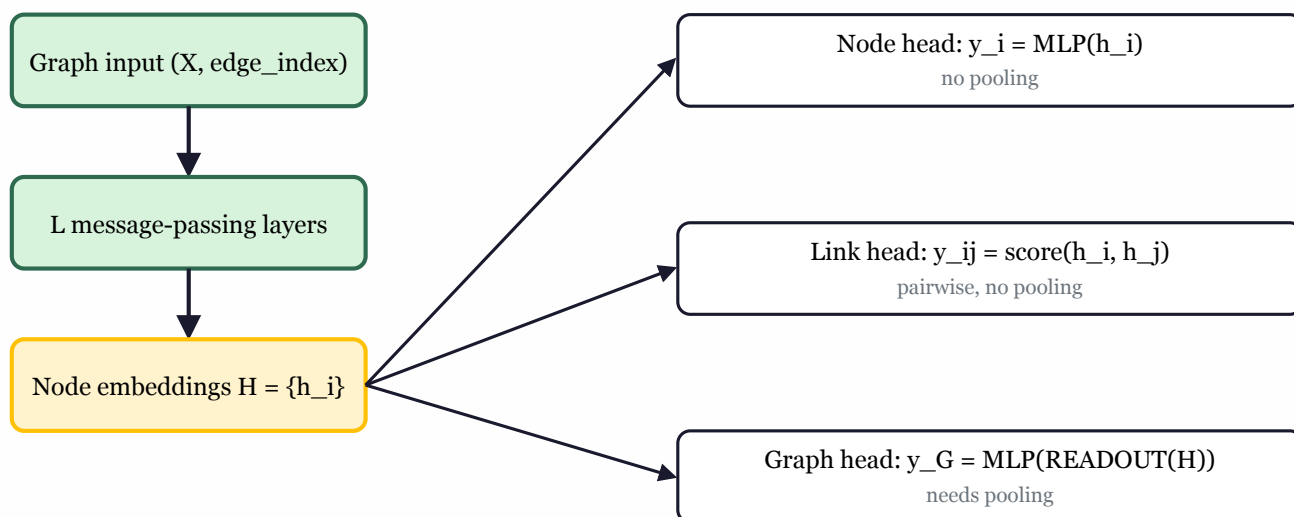
The Three Prediction Tasks

Lesson 1 introduced three levels of prediction — node, edge, and graph — and deliberately punted on the details: "the architecture stays mostly the same, only the final pooling/readout step changes." Lesson 2 gave you the tensors (X , $edge_index$) that a GNN actually consumes. This lesson closes the loop: given those tensors, what does the *output* side of the network look like for each task?

A shared trunk, three different heads

Every GNN we'll study in this course has the same basic shape. A stack of message-passing layers (the mechanics are Lesson 4's subject) transforms the raw node features X into learned node representations. We'll call these h_i — the hidden state, or embedding, of node i — to distinguish them from the raw input feature x_i they started as. After L layers, every node i has an embedding h_i that encodes not just its own features but information gathered from its local neighborhood.

What happens *next* is where node-, edge-, and graph-level tasks diverge. The encoder (the message-passing stack) is shared; only the head bolted onto the end changes.



Same encoder, three heads. Only the graph-level head needs to compress a variable-size set of node embeddings into one fixed-size vector.

Node-level: classify what's already there

If the target is a property of individual nodes — is this account fraudulent, which faction does this club member join, what part of speech is this token — the answer is already sitting at each node after L layers. You take h_i and pass it through a shared classifier (typically a small MLP or even a single linear layer) to get a per-node prediction:

```
y_i = MLP(h_i)    # applied independently to every node i
```

Key idea: Node classification needs *no pooling step at all*. The same classifier head is applied node-by-node, weight-shared exactly the way a CNN's final 1×1 convolution is applied pixel-by-pixel. This is the simplest of the three heads — everything interesting already happened inside the message-passing layers.

Edge-level: a pairwise scoring function

Link prediction — will these two nodes connect? what's the type of this relationship? — needs to combine *two* node embeddings into one prediction. This introduces something node classification never needed: a function that takes a *pair* of embeddings and produces a score.

```
y_ij = score(h_i, h_j)

# common choices for score():
#   dot product:      score(h_i, h_j) = h_i · h_j
```

```
# concat + MLP:      score(h_i, h_j) = MLP([h_i ; h_j])
# bilinear:          score(h_i, h_j) = h_i^T W h_j
```

The score is usually squashed through a sigmoid to get a probability that the edge exists, then trained against known edges (positive examples) and sampled non-edges (negative examples) — a detail we'll return to when we cover training strategies in Lesson 13.

Analogy: This is structurally the same move as a Siamese network comparing two embeddings for similarity, or the query-key dot product inside attention. The GNN's job is just to produce embeddings good enough that a simple pairwise function on top of them is informative.

Graph-level: everyone gets a vote

Graph classification — is this molecule toxic, will this program crash — needs a single vector representing the *whole* graph, but the encoder only ever produced one embedding per node, and graphs vary in node count. Something has to compress a variable-size set $\{h_1, \dots, h_n\}$ into one fixed-size vector before a classifier can touch it. That compression step is called **readout** (or pooling):

```
h_G = READOUT({h_i : i in V})      # e.g. sum, mean, or max over all nodes
y_G = MLP(h_G)
```

The simplest readouts are exactly the permutation-invariant aggregators you'd guess from Lesson 1's equivariance discussion — sum, mean, max over all node embeddings. More expressive options (attention-weighted pooling, hierarchical pooling that clusters nodes first) exist and get their own full treatment in **Lesson 12**; for now, the important point is structural, not which specific readout you pick.

Key idea: Graph-level prediction is the only one of the three tasks that strictly requires a permutation-*invariant* aggregation over all nodes. Node and edge tasks only ever need permutation-*equivariant* per-node or per-pair operations — recall the equivariant/invariant distinction from Lesson 1's glossary entries.

Same tensors, different labels

Tying this back to Lesson 2's plumbing: the input tensors (`X` , `edge_index`) are identical across all three tasks. What differs is only the shape of the training *label* and which head sits on top of the shared encoder:

Task	Label shape	Head	Pooling?
Node classification	one label per node	$\text{MLP}(h_i)$	No
Link prediction	one label per node pair	$\text{score}(h_i, h_j)$	No
Graph classification	one label per graph	$\text{MLP}(\text{READOUT}(H))$	Yes

Check your understanding

1. A GNN scores the fraud risk of every individual account node in a transaction graph. Which output-layer design fits this task?

Per-node MLP, no pooling

Graph-level MLP after pooling

Pairwise scorer over embeddings

Global readout then softmax

Fraud risk per account is a node-level target, so each node's embedding is passed through a shared classifier directly. No pooling step is involved — that's only needed once the target is a property of the whole graph.

2. Which output-head design is distinctive to link prediction, and not needed for node or graph classification?

A pairwise node-embedding scorer

A per-node softmax classifier

A permutation-invariant readout function

A stack of MLP layers

Link prediction combines *two* node embeddings (via a dot product, bilinear form, or concatenation into an MLP) into a single score for whether an edge exists. Neither the node head nor the graph head needs this pairwise combination step.

3. Predicting overall molecule solubility from atom-level embeddings requires which step that node classification skips entirely?

A readout that pools nodes

A pairwise edge score function

A shared per-node linear layer

A larger node feature matrix

Graph-level tasks must compress a variable number of node embeddings into one fixed-size graph vector via a permutation-invariant readout (sum, mean, max, or something

fancier — Lesson 12) before the final classifier can run. Node classification never needs this compression.

Primary source

Read: "A Gentle Introduction to Graph Neural Networks" (<https://distill.pub/2021/gnn-intro/>) by Sanchez-Lengeling et al. (Distill, 2021), **Section 4** ("Prediction tasks on graphs"). It walks through graph-level, node-level, and edge-level examples with interactive diagrams — molecule smell prediction, Zach's karate club, and scene-graph edge pruning respectively.

Questions about anything in this lesson? Ask your teacher — run `/teach` and ask about whatever is unclear. Next up: **Lesson 4 — Message Passing: How Nodes Talk to Neighbors**, where we open up what actually happens inside those "L message-passing layers" we've been treating as a black box.

← Lesson 2: Graph Representations in Practice

Lesson 4: Message Passing →

Message Passing: How Nodes Talk to Neighbors

Lesson 3 treated "L message-passing layers" as a black box that turns raw features X into embeddings H . This lesson opens the box. Message passing is the single mechanism underneath essentially every GNN you'll meet in this course — GCN, GraphSAGE, GAT, GIN are all specific choices of the functions inside the same three-step loop. Understand this lesson well and the rest of the course is mostly "which flavor of these three functions did they pick."

The gather → aggregate → update loop

One message-passing layer updates every node's representation using its current neighbors' representations. For a node i with neighborhood $N(i)$ (from the glossary), one layer does three things:

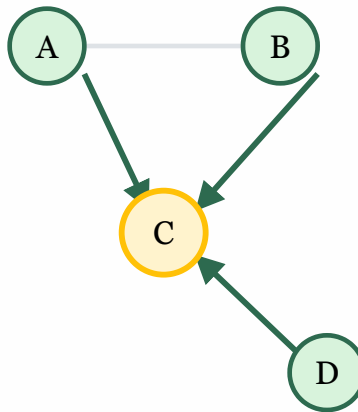
1. **Gather / message:** for each neighbor $j \in N(i)$, compute a message $m_{j \rightarrow i} = \text{MSG}(h_i, h_j, e_{ij})$ — a function of the receiving node's own state, the sending neighbor's state, and (optionally) the edge feature between them.
2. **Aggregate:** combine all incoming messages into one fixed-size vector: $a_i = \text{AGG}(\{m_{j \rightarrow i} : j \in N(i)\})$.
3. **Update:** combine the aggregated message with the node's own previous state to produce its new state: $h_i' = \text{UPD}(h_i, a_i)$.

Every node in the graph runs this same three-step recipe, in parallel, using the *same* learned MSG, AGG, and UPD functions (weight-sharing, just like a CNN kernel reused at every pixel). Stack L of these layers back to back and you get exactly the "L message-passing layers" box from Lesson 3.

Key idea: A message-passing layer is a local, weight-shared computation: it only ever looks at a node, its direct neighbors, and the edges between them. Nothing in the layer definition requires knowing the whole graph at once — which is exactly why this scales to graphs with billions of nodes, and why `edge_index` (Lesson 2) is all a layer ever needs to read.

Worked example: updating node C

Let's trace one layer by hand on the four-node graph from Lesson 2 (A–B, A–C, B–C, C–D). Node C has three neighbors: A, B, and D.



Node C gathers one message each from A, B, and D, then aggregates and updates.

```
# gather: one message per neighbor of C
m_A = MSG(h_C, h_A)
m_B = MSG(h_C, h_B)
m_D = MSG(h_C, h_D)

# aggregate: combine the (unordered) set {m_A, m_B, m_D}
a_C = AGG({m_A, m_B, m_D})    # e.g. sum, mean, or max
```

```
# update: fold the aggregate into C's own previous state
h_C' = UPD(h_C, a_C)
```

Meanwhile A, B, and D are running the exact same recipe over *their* own neighborhoods, at the same time. After one layer, every node's embedding has absorbed information from its immediate (1-hop) neighbors — what happens when you stack more than one layer is Lesson 5.

Why aggregation must be permutation-invariant

Notice that step 2 wrote AGG over a *set* $\{m_A, m_B, m_D\}$, not a list. That's deliberate. A node's neighborhood $N(i)$ has no canonical order — "A, B, D" and "D, B, A" describe the exact same neighbor set. The only thing that distinguishes A, B, and D from each other is *their own features*, carried inside each message; the order we happen to iterate over them while writing code is an implementation detail, not part of the graph.

Recall Lesson 1's permutation equivariance requirement: relabeling the nodes of a graph must not change what the GNN computes, only how the output is indexed. If AGG's result depended on the order its inputs were listed, then two people drawing the identical graph with a different node-labeling convention would get a different embedding for the same node — a GNN that isn't equivariant to a completely arbitrary choice (how you happened to number the nodes) is not usable as a general graph model.

Key idea: AGG must satisfy $\text{AGG}(\{m_A, m_B, m_D\}) = \text{AGG}(\{m_B, m_D, m_A\}) = \dots$ for every permutation of the same set. Functions that reduce a set to a fixed-size vector this way — sum, mean, max — are called **permutation-invariant aggregators**. This single constraint is what makes message passing well-defined on graphs at all.

Choosing an aggregator: sum vs. mean vs. max

Sum, mean, and max are all valid (permutation-invariant), but they preserve different information:

Aggregator	Keeps	Loses	Typical use
Sum	Neighbor count (degree) information	Scale grows unbounded with degree	GIN (Lesson 10) — maximizes expressiveness
Mean	A degree-normalized average signal	How many neighbors contributed	GCN (Lesson 7), GraphSAGE-mean
Max	The single most salient neighbor signal	Everyone else's contribution entirely	GraphSAGE-pool

Analogy: This is the same trade-off as choosing sum-pooling vs. average-pooling vs. max-pooling in a CNN. Sum keeps a signal proportional to "how much stuff is here," mean normalizes that away, and max keeps only the strongest activation. The graph setting is more consequential, though: unlike a CNN's fixed 3×3 window, a node's neighborhood size (its degree) varies wildly across the graph, so the choice of aggregator interacts directly with degree variance.

A quick reference card capturing this whole framework in clean notation and pseudocode lives at reference/message-passing-cheatsheet.html (../reference/message-passing-cheatsheet.html) — worth bookmarking, since GCN, GraphSAGE, GAT, and GIN are all just different (MSG, AGG, UPD) triples plugged into this same loop.

Check your understanding

1. Node C has neighbors A, B, and D. If you swap the order in which their messages are listed before aggregating, what should happen to C's updated embedding?

The result stays unchanged

The result becomes random

The result flips sign

The result loses information

Sum, mean, and max are permutation-invariant: reordering the same set of neighbor messages doesn't change the aggregated output. This is required so the computation doesn't secretly depend on an arbitrary node-listing order.

2. Which of the following aggregation functions is *not* permutation-invariant over a neighbor set?

Concatenation in listed order

Element-wise sum of messages

Element-wise mean of messages

Element-wise max of messages

Concatenating messages in whatever order you happened to iterate over them produces a different vector for a different ordering of the same neighbor set — it's *order-dependent*, not invariant. Sum, mean, and max all give the same result regardless of order.

3. A GCN-style mean aggregator divides the summed neighbor messages by degree. What does this primarily correct for?

Nodes with very different degrees

Nodes with identical feature vectors

Edges with missing edge features

Layers with too few parameters

Mean aggregation normalizes by neighbor count, so a high-degree node doesn't simply dominate by summing over more terms. Without this normalization, nodes with widely varying degree would produce aggregate messages on very different scales.

Primary source

Read: "A Gentle Introduction to Graph Neural Networks" (<https://distill.pub/2021/gnn-intro/>) by Sanchez-Lengeling et al. (Distill, 2021), **Section 6** ("GNN predictions by pooling information"), for the interactive gather/aggregate walkthrough. For the formal treatment with full notation, see William L. Hamilton's "Graph Representation Learning" (https://www.cs.mcgill.ca/~wlh/grl_book/), **Section 5.1** ("Neural Message Passing").

Questions about anything in this lesson? Ask your teacher — run `/teach` and ask about whatever is unclear. Next up: **Lesson 5 — Stacking Layers and the Receptive Field**, where we see what happens when you chain several of these layers together.

Stacking Layers and the Receptive Field

Lesson 4 defined a single message-passing layer: gather from direct neighbors, aggregate, update. Lesson 3 casually referred to "L message-passing layers" without saying what L actually buys you. This lesson answers that: depth in a GNN controls exactly how far information can travel across the graph, in a way that maps directly onto something you already understand from CNNs — the receptive field.

One layer, one hop

After a single message-passing layer, node i 's new embedding $h_i^{(1)}$ is a function of $h_i^{(0)}$ and $\{h_j^{(0)} : j \in N(i)\}$ — itself and its immediate (1-hop) neighbors, and nothing else. A node two hops away has had zero influence on $h_i^{(1)}$, because it never appeared in any message i received.

Two layers, two hops

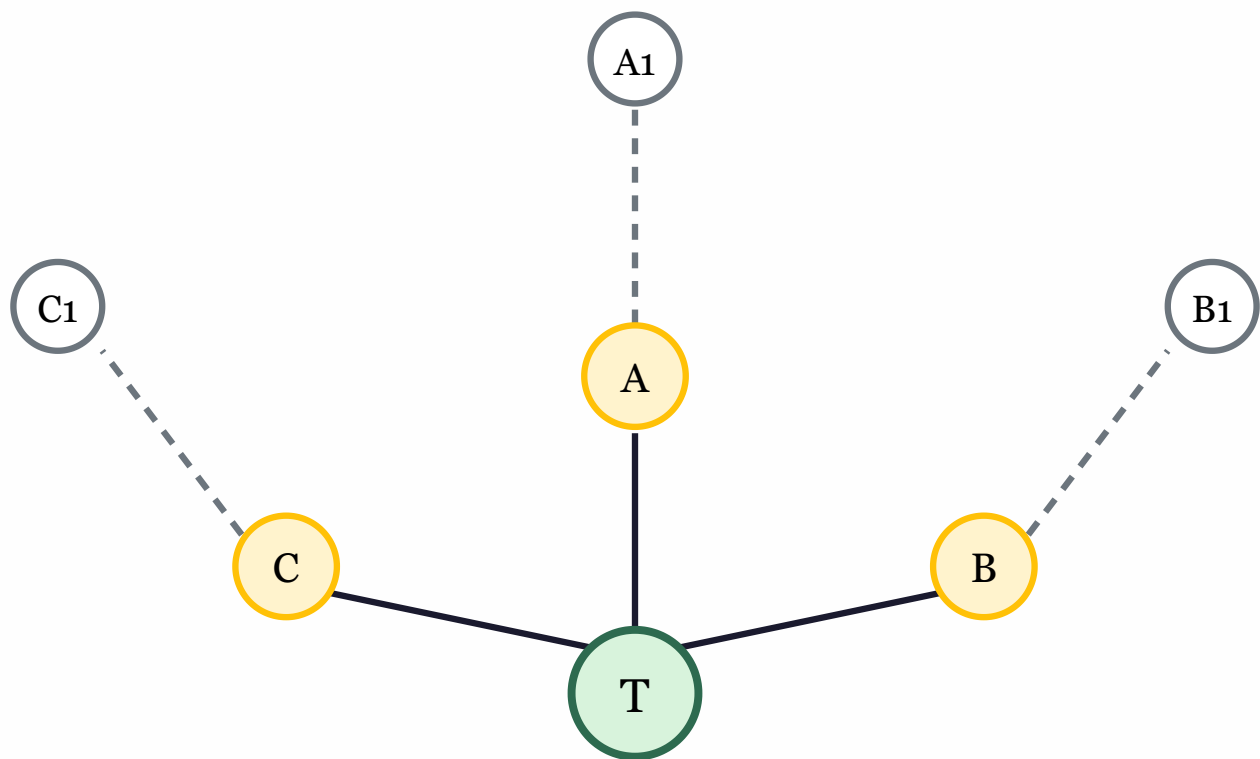
Now stack a second layer. Computing $h_i^{(2)}$ requires the layer-1 embeddings of i 's neighbors: $\{h_j^{(1)} : j \in N(i)\}$. But each of those $h_j^{(1)}$ was itself built from j 's own 1-hop neighbors in the previous layer — which includes nodes that are two hops away from i . Information from a 2-hop neighbor reaches i indirectly, by first being folded into an intermediate neighbor's embedding, then relayed to i .

Key idea: After k layers, $h_i^{(k)}$ is a function of every node within i 's **k -hop neighborhood** — and, generically, of nothing farther away. This set of nodes that can possibly influence $h_i^{(k)}$ is called i 's **receptive field**, borrowing the term directly from CNNs: stacking k convolutional layers with 3×3 kernels

grows a pixel's receptive field by one ring each time, exactly the way stacking k message-passing layers grows a node's receptive field by one hop each time.

Worked example: a 2-layer receptive field

Take a small tree rooted at a target node T , with three 1-hop neighbors (A , B , C), each of which has its own further neighbor (A_1 , B_1 , C_1) two hops from T :



Solid edges: T 's 1-hop neighbors (green target, amber ring). Dashed edges: 2-hop neighbors reached only through A , B , or C (white ring). A 2-layer GNN's receptive field for T covers all seven nodes; a 1-layer GNN's covers only T , A , B , C .

```
# layer 1: every node absorbs its direct neighbors
h_A^(1) depends on {A, T, A1}
```

```
h_T^(1) depends on {T, A, B, C}           # T's receptive field: 1 hop

# layer 2: T absorbs its neighbors' layer-1 states
h_T^(2) depends on h_A^(1), h_B^(1), h_C^(1), h_T^(1)
           = depends on {T, A, B, C, A1, B1, C1}   # T's receptive field:
```

Notice A_1 never appears directly in any message T receives — it enters only because it was already baked into $h_A^{(1)}$ by the time T reads it. This chaining is exactly why depth matters: each additional layer lets information relay one hop further before it reaches T .

The computation graph

Unrolling this by hand, as above, produces a tree rooted at the target node — T 's **computation graph**. This is a genuinely useful way to think about any single node's forward pass: it's a k -layer-deep tree where the root is the node you care about, and the leaves are its k -hop neighbors. (Distill's "Understanding Convolutions on Graphs" frames the same idea as repeatedly applying a 1-hop localized convolution K times to grow the effective receptive field to K hops — identical idea, spatial-convolution framing.)

How deep should you go?

More hops sounds strictly better — more context should mean a better-informed prediction — but two practical constraints push back:

- **Cost.** A node's k -hop receptive field can grow exponentially with the graph's branching factor (average degree). For a node with degree d , a naive k -hop expansion touches up to $O(d^k)$ nodes — this is why GraphSAGE (Lesson 8) samples a fixed number of neighbors per layer rather than using the full neighborhood.

- **Diminishing (and eventually harmful) returns.** Real-world graphs are often "small-world": most node pairs are separated by surprisingly few hops, so a handful of layers (commonly 2–4) already covers most of the graph. Pushing depth further doesn't just get expensive — it actively degrades quality, which is Lesson 6's entire subject.

Analogy: This mirrors the CNN depth-vs-receptive-field trade-off you already know, but with a twist: an image's grid has a fixed branching factor (a 3×3 kernel always looks at 8 neighbors), so a CNN's receptive field grows predictably and linearly in area. A graph's degree varies node to node, so the receptive field's growth rate is irregular and can explode for high-degree "hub" nodes — something images never have to deal with.

Check your understanding

1. In the worked tree example, why does node A1's feature influence h_T after 2 layers but not after 1 layer?

It reaches T only via A's layer-1 state

It reaches T only via T's own features

It reaches T only via a self-loop edge

It reaches T only via the readout step

A1 is two hops from T. After layer 1, A1's feature has only been folded into A's embedding. Only when T aggregates A's layer-1 embedding during layer 2 does A1's information indirectly reach T.

2. Stacking a GNN to exactly k layers grows each node's receptive field to cover which set of nodes?

Every node within k hops

Every node in the graph

Only the k nearest edges

Only nodes with k neighbors

Each layer relays information one additional hop, so after k layers a node's embedding is a function of (generically) every node reachable within k hops — not the whole graph, unless the graph's diameter happens to be $\leq k$.

3. Why can a node's k-hop receptive field grow far more unpredictably than a CNN pixel's receptive field?

Node degree varies across the graph

Node features vary across the graph

Edge weights vary across the graph

Layer count varies across the graph

A CNN kernel always has a fixed, uniform branching factor (e.g., 8 neighbors per pixel), so receptive field growth is predictable. A graph's degree distribution is irregular, so high-degree "hub" nodes can pull in vastly more nodes per hop than low-degree ones.

Primary source

Read: William L. Hamilton's "Graph Representation Learning"

(https://www.cs.mcgill.ca/~wlh/grl_book/), **Section 5.2** ("Generalized Neighborhood Aggregation"), for the formal treatment of stacking layers. For the CNN-style framing of receptive-field growth via repeated localized convolutions, see Sanchez-Lengeling et al.'s "Understanding Convolutions on Graphs" (<https://distill.pub/2021/understanding-gnns/>) (Distill, 2021), **Section 6**.

Questions about anything in this lesson? Ask your teacher — run `/teach` and ask about whatever is unclear. Next up: **Lesson 6 — Over-smoothing and the Depth Problem**, where we see why "just add more layers" backfires.

← Lesson 4: Message Passing

Lesson 6: Over-smoothing and the Depth Problem →

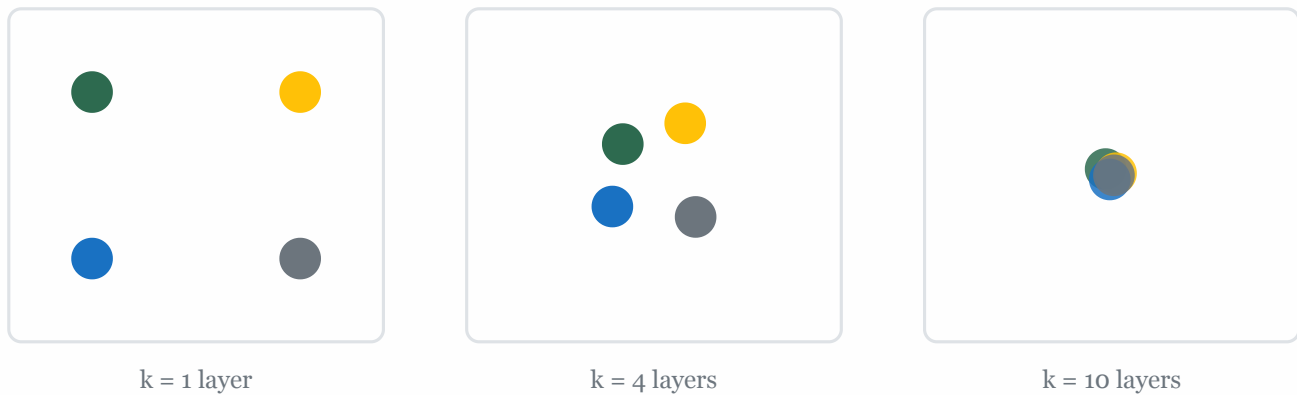
Over-smoothing and the Depth Problem

Lesson 5 established that stacking k layers grows a node's receptive field to k hops, and made it sound like a free lunch: more layers, more context, better predictions. In practice, GNN accuracy on node classification typically *peaks* at 2–4 layers and then falls off a cliff as you add more. This lesson explains why, and what practitioners actually do about it.

The core failure mode: representational collapse

Recall from Lesson 4 that a GCN-style update averages a node's own features with its neighbors' features at every layer. Li, Han & Wu (2018) made a sharp observation about what this operation actually is: graph convolution of this form is a special case of **Laplacian smoothing**, the classical technique for denoising a signal defined over a graph by repeatedly replacing each node's value with a (weighted) average of itself and its neighbors.

Laplacian smoothing is exactly what you want *once* — it's a low-pass filter that blurs out noise while keeping the broad structure of the signal. But apply it over and over, and it keeps removing high-frequency detail every single time. On a connected graph, repeated smoothing provably drives every node's representation toward the same fixed point (a value proportional to node degree, independent of the original input features) as the number of layers grows — a mathematical convergence result, not just an empirical tendency. This is **over-smoothing**: node embeddings that started out distinguishable become progressively harder, then impossible, to tell apart as depth increases.



Four nodes' embeddings, projected to 2D, as depth increases. Each repeated round of neighbor-averaging squeezes the points closer together until they're nearly indistinguishable.

Analogy: This is the graph-signal equivalent of repeatedly Gaussian-blurring an image. One blur pass removes sensor noise and looks fine. Apply the same blur kernel a hundred times and the image converges to a flat, featureless gray — every pixel's value approaches the same global average, and the picture (the useful signal) is gone. GCN layers are, structurally, a blur kernel defined over the graph instead of the pixel grid.

Why this hurts learning

Node classification depends on nodes being distinguishable: a classifier can only separate class A from class B if their embeddings actually differ. Once over-smoothing sets in, h_i stops carrying much information about node i specifically and starts reflecting mostly the local graph structure (degree, connected component) — the same collapse for every node regardless of its original label. Empirically, this is exactly what's observed: GCN accuracy on citation-network node classification tends to peak around 2–3 layers and degrades sharply beyond roughly 6–10, well before the receptive field has even reached most of the graph.

Key idea: Depth and over-smoothing are in tension with the receptive-field argument from Lesson 5. More layers give a node access to more of the graph,

but every additional layer is also another round of smoothing. The practical range where "more context" outweighs "more blurring" is usually narrow — a handful of layers, not dozens.

Mitigations

None of these change the fact that message passing smooths; they change how much, how fast, or add a path around it:

Mitigation	Idea
Fewer layers	Use only as many layers as the task's effective receptive field needs (Lesson 5) — often 2–4 is enough given small graph diameters, so simply not over-building depth sidesteps the problem.
Residual / skip connections	Re-inject an earlier-layer (or the original, layer-0) embedding at each layer, e.g. GCNII's initial-residual design, so the identity signal isn't fully washed out by repeated smoothing.
Normalization — PairNorm	Rescale node embeddings after each layer so the total pairwise distance between all nodes is held roughly constant, directly counteracting the tendency of embeddings to converge (Zhao & Akoglu, 2019).
Stochastic edges — DropEdge	Randomly remove a fraction of edges at every training step, which both slows the rate of smoothing and acts as data augmentation against overfitting (Rong et al., 2020).

Key idea: Notice the family resemblance to ResNets. Over-smoothing in deep GNNs and vanishing/degrading signal in very deep plain CNNs share the same shape of problem — each layer's transformation erodes something useful about the input — and skip connections are a shared-lineage fix in both settings. The graph-specific twist is that the eroding force here is smoothing (a low-pass filter), not just optimization difficulty.

In practice, most GNNs you'll encounter default to 2–4 message-passing layers and only reach for these tricks when a task genuinely needs longer-range dependencies than a shallow model's receptive field can cover. A common complementary move is to add ordinary (non-message-passing) MLP layers before or after the GNN stack — more capacity, without more smoothing.

Check your understanding

1. As a GCN is stacked to many layers, repeated neighbor-averaging acts like which image-processing operation applied over and over?

Repeated Gaussian blurring

Repeated edge detection

Repeated contrast sharpening

Repeated color inversion

Li, Han & Wu (2018) show that graph convolution is a form of Laplacian smoothing. Applying it repeatedly acts like a low-pass filter that blurs away the high-frequency signal distinguishing neighboring nodes — the graph analogue of repeated Gaussian blurring on an image.

2. Which symptom is the hallmark of over-smoothing in a deep GNN?

Node embeddings become nearly identical

Node embeddings become extremely sparse

Node embeddings gain extra dimensions

Node embeddings lose their gradients

Over-smoothing is representational collapse: repeated neighbor-averaging drives node embeddings toward a shared fixed point, so nodes that started out distinguishable become nearly identical, wrecking any classifier trying to tell them apart.

3. Which technique specifically adds a connection back to a node's earlier-layer (or initial) representation to counter over-smoothing?

Residual or skip connections

Larger learning rate schedules

Wider hidden layer dimensions

Additional dropout on inputs

Residual/skip connections (e.g. GCNII's initial-residual design) re-inject an earlier-layer or original node representation at each layer, preventing the signal from being fully washed out by repeated smoothing — complementary to PairNorm (renormalizing embeddings) and DropEdge (randomly removing edges to slow smoothing).

Primary source

Read: Qimai Li, Zhichao Han & Xiao-Ming Wu, "Deeper Insights into Graph Convolutional Networks for Semi-Supervised Learning" (<https://arxiv.org/abs/1801.07606>) (AAAI 2018, arXiv:1801.07606) — the paper that formalized graph convolution as Laplacian smoothing and named the over-smoothing problem. For the applied framing and empirical depth-vs-accuracy curves, see Stanford CS224W (<https://cs224w.stanford.edu/>), Lecture 6 ("Theory of Graph Neural Networks" / expressive power and over-smoothing).

Questions about anything in this lesson? Ask your teacher — run `/teach` and ask about whatever is unclear. Next up: **Lesson 7 — GCN: Spectral Roots, Spatial Practice**, where we finally derive the specific (MSG, AGG, UPD) triple that defines the original GCN, and see where its self-loops and degree-normalization actually come from.

← Lesson 5: Stacking Layers

Lesson 7: GCN: Spectral Roots, Spatial Practice →

GCN: Spectral Roots, Spatial Practice

Lessons 4–6 gave you the generic message-passing recipe — gather messages from a node's neighborhood, aggregate them with a permutation-invariant function, update the node's representation — and showed what stacking that recipe does to a node's receptive field. Now we specialise. The **Graph Convolutional Network (GCN)** of Kipf & Welling (2017) was the architecture that made this recipe concrete and fast enough to matter, and it did so by way of a surprising detour through spectral graph theory.

The limitation it solves: convolution needs a "graph Fourier transform"

Before GCN, the natural way to define "convolution on a graph" was to borrow directly from signal processing. A CNN's convolution is a multiplication in the *frequency* domain, made cheap by the Fast Fourier Transform. Graphs have an analogous frequency basis: the eigenvectors of the **graph Laplacian** $L = D - A$. A "spectral graph convolution" filters a signal x by transforming it into this eigenbasis, multiplying by a learned filter g_θ , and transforming back.

This works mathematically, but it's impractical:

- **Eigendecomposition is $O(|V|^3)$** — infeasible for graphs with more than a few thousand nodes.
- **The learned filter is global**, not localized to a neighborhood — every node's filter depends on the entire spectrum, which also means the number of filter parameters scales with graph size and can't transfer to a differently-sized graph.

Kipf & Welling's contribution was to approximate this spectral filter so aggressively that both problems vanish, leaving behind a rule that looks purely spatial — a weighted average over a node's immediate neighbors, exactly the shape of the message-passing framework from Lesson 4.

From spectral filter to spatial rule

The derivation goes through two approximation steps:

1. **Chebyshev truncation.** Approximate the spectral filter g_θ with a truncated Chebyshev polynomial expansion of the Laplacian, up to order K . This avoids the eigendecomposition entirely — polynomials of L can be applied directly — and a K -th order polynomial is exactly K -hop localized, since $(L^K)_{ij}$ is nonzero only when i and j are within K hops.
2. **First-order truncation ($K=1$), linearized.** GCN takes this to the extreme: $K=1$, and even the single remaining coefficient is fixed rather than learned. What's left is a filter that only looks at *direct* neighbors per layer — depth, not filter order, is what extends the reach, exactly as Lesson 5 described for the receptive field.

After this collapse, and after a further trick to keep the eigenvalues of the propagation operator numerically well-behaved (the **renormalization trick**), the entire derivation reduces to one equation applied at every layer:

The GCN propagation rule:

$$H^{(l+1)} = \sigma(\tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2} H^{(l)} W^{(l)})$$

where $\tilde{A} = A + I_N$ (adjacency matrix *with self-loops added*), $\tilde{D}$ is the degree matrix of $\tilde{A}$ (so $\tilde{D}_{ii} = \sum_j \tilde{A}_{ij}$), $H^{(l)}$ is the matrix of node representations at layer l ($H^{(0)} = X$), $W^{(l)}$ is a learned weight matrix shared across all nodes, and σ is a nonlinearity like ReLU.

Self-loops ($\tilde{A} = A + I$) are what let a node's own features survive into the aggregate — recall from Lesson 2 that this was foreshadowed as "we'll see exactly why this matters when we derive the GCN update rule." Without them, a node's representation at each layer would depend only on its neighbors, never on itself.

Reading the rule as message passing

Mapped onto the gather $\rightarrow$ aggregate $\rightarrow$ update vocabulary from Lesson 4:

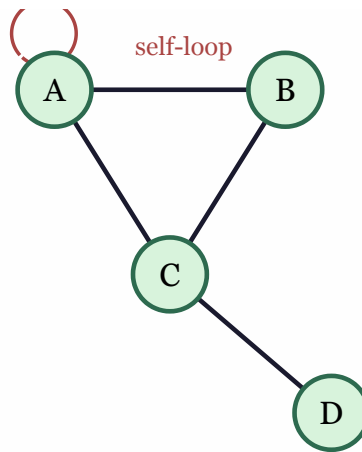
- **Message:** each neighbor j sends its current representation h_j , linearly transformed by the shared W .
- **Aggregate:** a *weighted sum* over neighbors (including the node itself, via the self-loop), where the weight between i and j is $\tilde{D}_{ii}^{-1/2} \tilde{A}_{ij} \tilde{D}_{jj}^{-1/2}$.
- **Update:** apply the nonlinearity σ to the aggregated result.

The key design choice to notice: the aggregation weights are **fixed by graph structure** (they depend only on degrees), not learned or content-dependent. Every neighbor's contribution is determined the moment you know the graph — GCN doesn't ask "how relevant is this neighbor's *content*?" the way an attention-based architecture will in Lesson 9. It only asks "how many neighbors do you and I each have?"

Analogy: $\tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2}$ is a symmetric normalization — it downweights a message twice: once for the sender's degree, once for the receiver's degree. A high-degree "hub" neighbor gets less weight per edge (its influence is spread across many neighbors), and a node with many neighbors discounts each incoming message more (it has more messages to average over). It's the graph analogue of dividing by fan-in *and* fan-out.

Worked example: one GCN update, by hand

Reuse the four-node graph from Lesson 2 (A–B, A–C, B–C, C–D), and compute the aggregation weights feeding into node A after adding self-loops:



Degrees before self-loops: A=2, B=2, C=3, D=1. With self-loops added: A=3, B=3, C=4, D=2.

Node A's neighbors-with-self-loop are {A, B, C}, with degrees (post self-loop) $\tilde{D}_{AA}=3$, $\tilde{D}_{BB}=3$, $\tilde{D}_{CC}=4$. The normalized weight for each edge is $1/\sqrt{(\tilde{D}_{ii}\tilde{D}_{jj})}$:

$$\text{weight}(A \leftarrow A) = 1/\sqrt{(3 \times 3)} = 0.333 \quad (\text{self-loop term})$$

$$\text{weight}(A \leftarrow B) = 1/\sqrt{(3 \times 3)} = 0.333$$

$$\text{weight}(A \leftarrow C) = 1/\sqrt{(3 \times 4)} = 0.289$$

$$h_{A'} = \sigma(W \cdot (0.333 \cdot h_A + 0.333 \cdot h_B + 0.289 \cdot h_C))$$

Notice node D never enters A's update directly — it's two hops away. That's the receptive-field argument from Lesson 5 again: one GCN layer only reaches 1-hop neighbors, so D's influence on A requires stacking a second layer.

Transductive by default

The propagation rule as written bakes the graph's degree matrix $\tilde{D}$ and adjacency $\tilde{A}$ directly into every forward pass. That has a real consequence: the original GCN is a **transductive** method. It's trained and evaluated on *one fixed graph* — typically full-batch, using the entire normalized adjacency matrix at once. If a new node arrives after training, there's no defined procedure for folding it in without

recomputing the normalization and, in the full-batch formulation, rerunning the model over the whole graph again.

Key idea: "Transductive" doesn't mean GCN can't generalize at all — it generalizes across *unlabeled nodes already present in the training graph* (that's the "semi-supervised" in the paper's title). What it doesn't natively support is generalizing to *nodes that didn't exist at training time*. That distinction becomes central in Lesson 8, and matters directly for domains like fraud detection where new accounts and transactions appear continuously.

Check your understanding

1. In the GCN propagation rule, why are self-loops added to A before normalization?

So a node's own features get included

So every edge receives a doubled weight

So the graph becomes a complete graph

So gradients bypass the softmax normalization step

Without $\tilde{A} = A + I$, the aggregation would sum only over a node's neighbors and never include the node's own current representation, so each update would discard the node's prior state entirely.

2. GCN's degree normalization $\tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}$, compared to unnormalized A , mainly prevents what?

High-degree nodes dominating the aggregated signal

Low-degree nodes dominating the aggregated signal

Self-loops dominating the aggregated update signal

Attention weights dominating the aggregated signal

Without normalization, a node with many neighbors would sum up many unweighted messages, causing its representation's magnitude to grow with degree. Dividing by $\sqrt{(\text{degree})}$ on both sides keeps the aggregated scale comparable across nodes regardless of how many neighbors they have.

3. As originally formulated, is GCN transductive or inductive, and why?

Transductive — it needs the full fixed graph structure

Inductive — it generalizes directly to unseen new graphs

Transductive — it needs labeled data for every node

Inductive — it samples fixed-size neighborhoods like GraphSAGE does

The propagation rule bakes the specific graph's normalized adjacency matrix into every forward pass. It's trained and evaluated on one fixed graph, and there's no built-in mechanism for folding in nodes that weren't present at training time.

Primary source

Read: Kipf & Welling, "Semi-Supervised Classification with Graph Convolutional Networks" (<https://arxiv.org/abs/1609.02907>) (2017), Sections 2–3 for the spectral-to-spatial derivation and the renormalization trick. For the spectral-vs-spatial framing in more visual detail, see Distill's "Understanding Convolutions on Graphs" (<https://distill.pub/2021/understanding-gnns/>), §5.

Questions about anything in this lesson? Ask your teacher — run `/teach` and ask about whatever is unclear. Next up: **Lesson 8 — GraphSAGE: Inductive Learning and Sampling**, where we'll see the architecture explicitly built to handle unseen nodes.

← Lesson 6: Over-smoothing and the Depth Problem

Lesson 8: GraphSAGE →

GraphSAGE: Inductive Learning and Sampling

Lesson 7 ended on a limitation: GCN's propagation rule bakes the specific graph's normalized adjacency matrix into the model, so it's trained and evaluated on one fixed graph. **GraphSAGE** (SAmple and aggreGatE), from Hamilton, Ying & Leskovec (2017), was built explicitly to remove that constraint — and to make message passing tractable on graphs with millions of nodes, where even loading the full adjacency matrix into memory is a problem.

The limitation it solves: fixed-graph, full-batch training doesn't scale or generalize

Two separate problems compound in the original GCN formulation:

- **Scalability.** Full-batch GCN computes a forward pass over the *entire* graph at once. For a graph with a billion edges, that's a lot of memory and compute per gradient step, and it only gets worse as you stack more layers — the receptive field of Lesson 5 grows to cover most of the graph.
- **Generalization to new nodes.** Because GCN's aggregation weights are derived from the fixed $\tilde{D}$ and $\tilde{A}$ of one graph, there's no defined way to compute an embedding for a node that arrives after training without recomputing over the whole graph again.

GraphSAGE addresses both with the same idea: stop treating the aggregation as an operation over the whole graph, and instead treat it as a *function* — learned once, applied locally to any node's neighborhood, on demand.

The core mechanism: sample, then aggregate

At each layer k , instead of gathering messages from *all* of a node's neighbors, GraphSAGE draws a fixed-size uniform sample S_k of neighbors (e.g. sample 10, even if the node has 500). The forward pass, for a node v at depth K , is:

GraphSAGE forward propagation (per node v , layer $k = 1..K$):

$$h_v^0 = x_v$$

$$h_{N(v)}^k = \text{AGGREGATE}_k(\{ h_u^{k-1} : u \in \text{sample}(N(v)) \})$$

$$h_v^k = \sigma(W^k \cdot \text{CONCAT}(h_v^{k-1}, h_{N(v)}^k))$$

$$h_v^k = h_v^k / \|h_v^k\|_2 \text{ (L2-normalize)}$$

Two details worth flagging against the message-passing template from Lesson 4:

- **Concatenation, not fixed weighted sum.** GCN mixes the self-term into the same weighted average as neighbors (via the self-loop). GraphSAGE instead *concatenates* the node's previous representation with the aggregated neighbor representation, then applies a learned W — giving the update more freedom to weigh "self" against "neighbors" separately, closer to a residual/skip connection.
- **Sampling makes cost independent of degree.** Since S_k has a fixed size, the compute and memory cost of one node's forward pass is bounded by K and the sample sizes, not by the graph's actual degree distribution — crucial for graphs with high-degree hub nodes.

Three aggregator choices

AGGREGATE_k must still be permutation-invariant (Lesson 4) since a sampled neighbor set has no inherent order. The paper evaluates three options:

- **Mean aggregator:** elementwise mean of the sampled neighbor vectors — the simplest option, structurally close to GCN's rule but without degree-based weighting.

- **LSTM aggregator:** feed the neighbor vectors through an LSTM for more expressive power. But an LSTM is not inherently permutation-invariant — the paper's workaround is to feed neighbors in a *random* order each time, adapting a sequence model to an unordered set.
- **Pooling aggregator:** transform every neighbor independently through a small MLP, then take an elementwise max: $\text{aggregate}_{\text{pool}} = \max(\{ \sigma(W_{\text{pool}} h_u + b) : u \in N(v) \})$. This is a direct ancestor of the max-based expressiveness argument you'll meet again in Lesson 10.

What's new: learning a function, not a lookup table

This is the conceptual heart of the paper. Earlier node-embedding methods (and GCN, in effect) learn something closer to a table of per-node vectors, fit to one specific graph. GraphSAGE instead learns the **aggregator functions and weight matrices W^k** — objects that don't reference any particular node's identity or the graph's specific size. Once learned, that same function can be applied to a node's neighborhood *regardless of whether that node existed during training*, as long as it has features and edges to sample from.

Analogy: GCN is like memorizing a lookup table of answers for the students in one specific classroom. GraphSAGE is like learning a general study method — a new student who walks in later (a new node, with features and neighbors) can still apply the same method to produce a sensible answer, without the table being recomputed.

Inductive by construction

This is why GraphSAGE is the canonical **inductive** GNN: it generalizes to nodes, and even entire graphs, unseen at training time, as long as they come with node features. That's a strict requirement, though — GraphSAGE needs meaningful node features to work from (e.g. profile attributes, transaction amounts). A method

relying purely on learned per-node identity embeddings, with no features at all, has nothing to compute for a brand-new node.

Key idea: Transductive vs. inductive is really a question of *what the model has learned to depend on*. GCN's learned parameters are entangled with one fixed graph's structure. GraphSAGE's learned parameters — the aggregator and W^k matrices — are graph-agnostic functions applied locally. This is exactly the property you'd want for a fraud graph where new accounts and transactions appear every hour and you can't afford to retrain (or reprocess the entire graph) before scoring them.

Training without labels: the unsupervised objective

GraphSAGE can be trained with ordinary supervised loss (e.g. cross-entropy on node labels) exactly like GCN. But the paper also proposes an unsupervised, graph-structure-based loss for settings without labels, based on the same intuition as word2vec-style embeddings: nodes that co-occur on short random walks should end up with similar embeddings, and randomly sampled distant nodes should not:

```
J(z_u) = -log(σ(z_u · z_v)) - Q · E[log(σ(-z_u · z_vn))]  
# v: a node near u on a random walk  
# vn: a "negative" node sampled roughly at random
```

This will resurface in Lesson 14 when we look at node embedding methods that predate GNNs entirely.

Check your understanding

1. GraphSAGE controls computation cost per node primarily by doing what?

Sampling a fixed number of neighbors

Sampling a fixed number of layers

Caching a fixed number of embeddings

Pruning a fixed number of edges

At each layer, GraphSAGE draws a fixed-size uniform sample of a node's neighbors rather than aggregating over all of them, so per-node cost stays bounded regardless of the node's actual degree.

2. What makes GraphSAGE inductive rather than transductive?

It learns aggregator functions, not embeddings

It learns node embeddings, not functions

It learns the full adjacency matrix

It learns attention weights per edge

GraphSAGE's learned parameters are the aggregator and weight matrices — graph-agnostic functions that can be applied to any node's local neighborhood, including nodes that didn't exist during training, as long as they have features.

3. Applying an LSTM aggregator to a node's neighbors requires which extra step to fit the permutation-invariance requirement from Lesson 4?

Randomly permuting neighbors before feeding the LSTM

Sorting neighbors by degree before the LSTM

Averaging neighbors together before the LSTM runs

Removing neighbors randomly before the LSTM runs

An LSTM processes its input as an ordered sequence, so it isn't naturally permutation-invariant. The paper's workaround is to feed the sampled neighbors in a random order each time, trading a clean theoretical guarantee for the LSTM's extra expressive power.

Primary source

Read: Hamilton, Ying & Leskovec, "Inductive Representation Learning on Large Graphs" (<https://arxiv.org/abs/1706.02216>) (2017). Focus on **Section 3** (the algorithm and aggregator architectures) and **Section 3.3** (the unsupervised and supervised loss functions).

Questions about anything in this lesson? Ask your teacher — run `/teach` and ask about whatever is unclear. Next up: **Lesson 9 — GAT: Attention on Graphs**, where neighbor weighting stops being fixed by degree and starts being learned from content.

GAT: Attention on Graphs

GCN weighs neighbors by degree. GraphSAGE's mean and pooling aggregators weigh sampled neighbors equally, or independently through a shared MLP before a symmetric max. Neither ever asks: *given what this specific neighbor's features actually say, how much should I trust it?* The **Graph Attention Network (GAT)**, from Veličković et al. (2018), answers that question by borrowing the attention mechanism you already know from sequence models and applying it edge by edge.

The limitation it solves: structural weights ignore content

GCN's aggregation weight between nodes i and j is $\tilde{D}_{ii}^{-1/2}\tilde{D}_{jj}^{-1/2}$ — fixed the moment you know the graph's degree sequence, regardless of what h_i and h_j actually contain. Two neighbors with identical degree get identical weight, even if one is far more relevant to the node's task than the other. This is a real cost: in a citation network, not every cited paper is equally informative about a paper's topic; in a transaction graph, not every counterparty is equally informative about whether an account is fraudulent.

Key idea: GAT replaces a *structure-determined* aggregation weight with a *learned, content-dependent* one. The weight a node assigns its neighbor is computed from both of their current feature vectors, not just from the graph's topology.

The core mechanism: self-attention over the neighborhood

For every edge (i, j) where $j \in N(i)$ (including i itself, via a self-loop, just as in GCN), GAT computes an unnormalized attention score:

Attention coefficient:

$$e_{ij} = \text{LeakyReLU}(a^T [Wh_i \parallel Wh_j])$$

where W is a shared learned linear transformation applied to every node's features, $\parallel$ is vector concatenation, and a is a single learned attention vector (shared across all nodes and edges) that scores how compatible the pair (Wh_i, Wh_j) is.

These raw scores aren't comparable in scale across different neighborhoods, so they're normalized with a softmax over each node's neighbor set — exactly the aggregation step from Lesson 4, now with learned rather than fixed weights:

Normalized attention weight:

$$\alpha_{ij} = \text{softmax}_j(e_{ij}) = \exp(e_{ij}) / \sum_{k \in N(i)} \exp(e_{ik})$$

$$h_i' = \sigma(\sum_{j \in N(i)} \alpha_{ij} Wh_j)$$

Notice the shape: this is still gather $\rightarrow$ aggregate $\rightarrow$ update. What changed is *only* how the aggregation weights are produced — a learned function of content, instead of a fixed function of degree.

Worked example: attention weights on a 3-node star

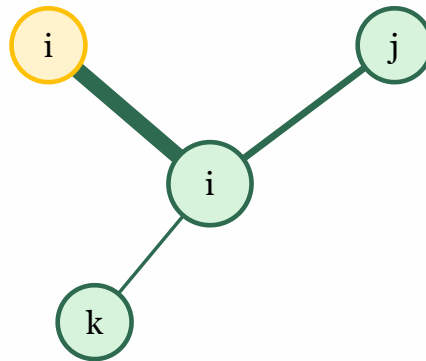
Take a toy case with 1-dimensional transformed features for clarity: node i has $Wh_i = 1.0$, and two neighbors j ($Wh_j = 0.5$, feature-aligned with i) and k ($Wh_k = -0.5$, feature-opposed to i). Suppose the learned attention vector reduces to simply summing the pair ($a = [1, 1]$), and self-loops are included:

$$e_{ii} = \text{LeakyReLU}(1.0 + 1.0) = 2.0 \quad (\text{self-loop})$$

$$e_{ij} = \text{LeakyReLU}(1.0 + 0.5) = 1.5$$

$$e_{ik} = \text{LeakyReLU}(1.0 + -0.5) = 0.5$$

$$\text{softmax}([2.0, 1.5, 0.5]) \rightarrow \alpha_{ii} \approx 0.547, \alpha_{ij} \approx 0.331, \alpha_{ik} \approx 0.122$$



Edge thickness $\propto$ attention weight. The self-loop ($\alpha=0.55$) and the feature-aligned neighbor j ($\alpha=0.33$) dominate the update; the feature-opposed neighbor k ($\alpha=0.12$) contributes little — a distinction GCN's degree-only weighting could never make, since all three would sit at whatever fixed weight the degrees imply.

Multi-head attention

A single attention mechanism can be unstable to learn and limited in what relationships it can capture, so GAT runs K independent attention heads in parallel, each with its own W^k and a^k . At hidden layers, their outputs are **concatenated**:

$$h_i' = \parallel_{k=1}^K \sigma \left(\sum_{j \in N(i)} \alpha_{ij}^k W^k h_j \right)$$

At the *final* output layer, concatenation would multiply the output dimension by K , which usually isn't what you want for a prediction head, so the heads are **averaged** instead:

$$h_i' = \sigma \left(\frac{1}{K} \sum_{k=1}^K \sum_{j \in N(i)} \alpha_{ij}^k W^k h_j \right)$$

Analogy: Multi-head attention here plays the same role it does in a Transformer: each head can specialize in a different notion of "relevance" (one head might key off feature similarity, another off something else entirely), and concatenating hidden-layer outputs lets downstream layers combine those perspectives, exactly as multi-head self-attention does over tokens.

What's new, and why it's efficient too

Two properties worth naming explicitly:

- **No graph-global computation.** Unlike the spectral derivation behind GCN, GAT's attention mechanism is defined purely locally — α is shared, but it's applied identically to every edge without reference to the graph's eigenstructure, degree matrix, or size. This also makes it naturally parallelizable across edges.
- **Shared, graph-agnostic parameters.** W and α don't depend on which node or graph they're applied to — only on the pair of feature vectors in front of them. That's the same property that made GraphSAGE inductive in Lesson 8, and the GAT paper explicitly validates this: it's tested on the same inductive PPI benchmark as GraphSAGE, in addition to transductive citation-network benchmarks. GAT's transductive-vs-inductive behavior depends on how it's trained and deployed, not on a structural constraint in the layer itself, the way GCN's fixed normalization is.

Check your understanding

1. What information does GCN's fixed normalization ignore that GAT's attention coefficients use?

The content of each neighbor's features

The degree of each neighbor's node

The distance of each neighbor's edge

The label of each neighbor's class

GCN's weight $\tilde{D}_{ii}^{-1/2}\tilde{D}_{jj}^{-1/2}$ depends only on degree. GAT computes e_{ij} from a learned function of $W h_i$ and $W h_j$ themselves, so two neighbors with the same degree can still receive very different attention weights.

2. Which operation in GAT ensures that a node's attention weights over its neighbors sum to one?

The softmax normalization over neighbors

The LeakyReLU activation over neighbors

The degree normalization over neighbors

The mean aggregation over neighbors

LeakyReLU produces the raw, unbounded score e_{ij} . The softmax, applied over all of node i 's neighbors (including itself via the self-loop), is what turns those raw scores into normalized weights α_{ij} that sum to one.

3. Why does GAT average attention heads at the final output layer instead of concatenating them, as it does at hidden layers?

Concatenating there would make output dimension head-count dependent

Averaging there always improves classification accuracy more substantially

Concatenating there would violate the permutation invariance property

Averaging there removes the need for a softmax

Concatenating K heads multiplies the representation width by K at every layer, which is fine internally but inconvenient at the final layer, where the output dimension should match the task (e.g. number of classes) rather than scale with the number of attention heads.

Primary source

Read: Veličković et al., "Graph Attention Networks" (<https://arxiv.org/abs/1710.10903>) (2018), Section 2 for the attention mechanism and Section 3 for how it compares to GCN and GraphSAGE, including the inductive PPI results.

Questions about anything in this lesson? Ask your teacher — run `/teach` and ask about whatever is unclear. Next up: **Lesson 10 — GIN: Maximising Expressiveness**, where we ask a harder question: not just how to weigh neighbors, but what any GNN can and can't tell apart at all.

GIN: Maximising Expressiveness

GCN, GraphSAGE, and GAT were each motivated by a practical concern — tractability, scalability, content-sensitivity. The **Graph Isomorphism Network (GIN)**, from Xu et al. (2019), asks a more theoretical question that turns out to have sharp practical consequences: *how much can a message-passing GNN, of any design, actually tell apart?* The answer reshapes how you should think about every architecture in this module — including the three you've just learned.

The limitation it solves: mean and max aggregation aren't injective

Every aggregator you've seen so far — GCN's normalized weighted sum, GraphSAGE's mean or max-pool, GAT's attention-weighted sum — compresses a *multiset* of neighbor feature vectors down to one vector. The question GIN's authors ask is: does that compression preserve enough information to tell different neighborhoods apart? For mean and max, the answer is provably no, in ways that don't depend on how big or well-trained the rest of the network is.

Two small examples make this concrete. Consider a node whose neighbors all share the same feature value 1, but the *count* of neighbors differs:

Neighbor multiset	mean	max	sum
{1, 1}	1	1	2
{1, 1, 1}	1	1	3

Mean and max produce *identical* outputs for a node with 2 identical neighbors and a node with 3 identical neighbors — that structural difference is simply gone before it ever reaches the update function. Sum preserves it. A second example shows

mean can also erase a difference in the neighbor *distribution*, not just the count: {1, 2} and {1, 1, 2, 2} both have mean 1.5, but sum gives 3 versus 6.

Key idea: Once an aggregator merges two different multisets into the same output, no amount of depth, width, or training in the layers that follow can ever recover the distinction — the information is destroyed at the aggregation step, not merely obscured. This is why aggregator choice is a hard ceiling on expressiveness, not just a design preference.

The core mechanism: sum aggregation, self-weighting, and an MLP

GIN's fix is to use sum aggregation — which the theory shows is capable of injectively encoding multisets under the right conditions — combined with a learnable weighting of the self-term and a multi-layer perceptron in place of the usual single linear transform + nonlinearity:

GIN update rule:

$$h_v^{(k)} = \text{MLP}^{(k)}((1 + \varepsilon^{(k)}) \cdot h_v^{(k-1)} + \sum_{u \in N(v)} h_u^{(k-1)})$$

where $\varepsilon^{(k)}$ is a scalar (learnable or fixed) that weighs the node's own previous representation relative to its summed neighbors, and $\text{MLP}^{(k)}$ is a small multi-layer perceptron, not just a single linear layer + σ .

Compare this to GCN's self-loop: GCN folds the self-term into the same normalized sum as every neighbor, with a weight fixed by degree. GIN instead gives the self-term its own tunable coefficient $(1+\varepsilon)$, separate from the neighbor sum — a more flexible, GAT-like idea of "how much should I weigh myself," but expressed as a single scalar rather than a full attention distribution.

Why an MLP instead of one linear layer? Because the theoretical claim requires being able to approximate *any* injective function over multisets, and a single linear layer followed by a fixed nonlinearity isn't expressive enough to guarantee that; a multi-layer perceptron, by the universal approximation theorem, is.

What's new: matching the Weisfeiler-Leman test

This is GIN's central theoretical result. The **Weisfeiler-Leman (WL) test** is a classical, purely combinatorial algorithm for graph isomorphism testing: every node starts with a color, and at each round, a node's color is re-hashed based on the *multiset* of its neighbors' colors. Two graphs are judged possibly-isomorphic if their final color histograms match after enough rounds; if the histograms differ, the graphs are provably non-isomorphic.

Message passing and the WL test have the same shape: both repeatedly recompute a node's state from a hashed/aggregated summary of its neighbors' states. Xu et al. prove two things:

- **The WL test is an upper bound.** No message-passing GNN, regardless of architecture, can distinguish two graphs that the WL test cannot distinguish.
- **GIN reaches that bound.** If the aggregation and update functions are injective (which sum + MLP can be, but mean/max cannot), the resulting GNN is exactly as powerful as the WL test at telling graphs apart.

Analogy: The WL test's color-hashing step is doing exactly what a message-passing layer does — summarize a neighborhood into a new label. GIN is, in effect, a WL test where the hand-designed "hash function" has been replaced by a learned, differentiable one (the MLP), trained end-to-end for the task instead of hand-built for isomorphism testing alone.

What this says about limitations, not just power

"As powerful as WL" is a ceiling, not just a floor. It implies that GIN — and by the bound above, *every* standard message-passing GNN, including GCN, GraphSAGE, and GAT — inherits the WL test's blind spots. There exist pairs of non-isomorphic graphs (for instance, certain pairs of regular graphs with matching degree sequences but different global structure) that WL cannot tell apart, and so no amount of extra layers, wider hidden dimensions, or more training data will let any of these architectures tell them apart either. We'll formalize this hierarchy in Lesson 15.

Where GIN fits: transductive, inductive, and readout

Like GraphSAGE and GAT, GIN's learned parameters (the MLPs and ε 's) are shared functions applied locally at every node, independent of a specific fixed graph — so it's inductive by construction in the same sense. In practice, GIN is almost always used for **graph-level** tasks (its motivating benchmarks are graph classification datasets), where every graph in the dataset is a separate training example and the model must generalize across graphs by design. Because sum aggregation is exactly what preserves multiset information, GIN also typically uses a *sum*-based readout to pool node embeddings into a graph-level representation — the same expressiveness argument applies one level up, a point we'll pick up in Lesson 12.

Check your understanding

1. Why can a GNN using mean aggregation fail to distinguish a node with two identical-feature neighbors from one with three identical-feature neighbors?

Mean normalizes by size, losing exact neighbor count

Sum normalizes by size, losing exact neighbor count

Max normalizes by size, losing exact neighbor count

Mean normalizes by count, losing exact neighbor size

Mean divides the sum by the number of neighbors, so two neighborhoods with the same repeated value but different sizes produce the same average — the count information is destroyed at the aggregation step, before any later layer can recover it.

2. According to Xu et al., what is the theoretical ceiling on how expressive any standard message-passing GNN can be at distinguishing non-isomorphic graphs?

The Weisfeiler-Leman graph isomorphism test

The spectral graph Laplacian eigendecomposition

The random walk transition matrix

The graph edit distance metric

Xu et al. prove that no message-passing GNN can distinguish two graphs the WL test cannot distinguish, and that GIN, with injective aggregation and update functions, achieves that same upper bound — making it exactly as powerful as WL, no more.

3. What role does GIN's learnable epsilon play in the update rule?

It weighs a node's own previous representation

It weighs each neighbor's own attention score

It weighs the total number of layers

It weighs the graph's overall readout vector

Epsilon scales the $(1+\epsilon) \cdot h_v$ term — the node's own previous representation — relative to the summed neighbor contributions, before the whole thing is passed through the MLP. It plays a role similar to GCN's self-loop, but as a separate, tunable coefficient rather than a fixed, shared normalization weight.

Primary source

Read: Xu et al., "How Powerful are Graph Neural Networks?"

(<https://arxiv.org/abs/1810.00826>) (2019). Focus on **Section 3** (the WL-equivalence theorems) and **Section 4.1** (the GIN architecture and the sum-vs-mean-vs-max comparison, illustrated in the paper's Figure 2–3).

Questions about anything in this lesson? Ask your teacher — run `/teach` and ask about whatever is unclear. Next up: **Lesson 11 — Architecture Comparison**, where we put GCN, GraphSAGE, GAT, and GIN side by side and build a decision framework for choosing between them.

Architecture Comparison

Four lessons, four architectures, each solving a specific limitation of the one before it: GCN made spectral convolution spatially tractable but stayed tied to one fixed graph; GraphSAGE traded fixed structural weighting for sampling and learned functions that generalize to unseen nodes; GAT traded fixed weighting for learned, content-dependent attention; GIN traded expressive weighting for guaranteed injectivity, reaching the theoretical ceiling of message-passing expressiveness. None of these is strictly "better" — each optimizes for a different constraint. This lesson builds the decision framework for picking one.

The framework isn't one axis

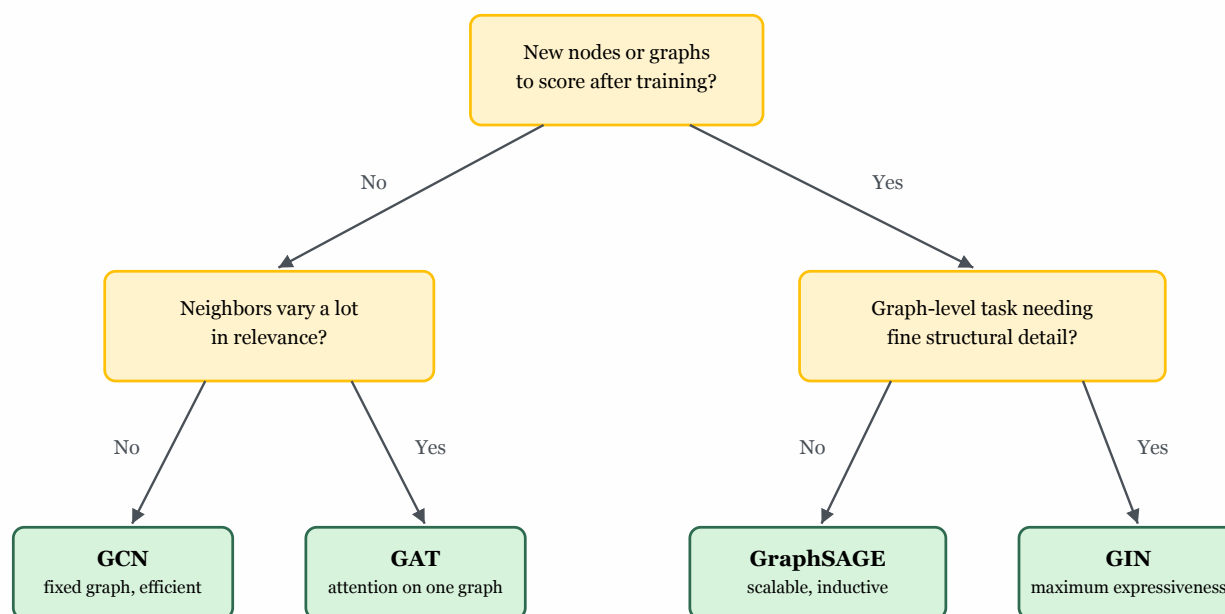
It's tempting to rank the four architectures on a single "power" scale. Resist that — the real decision runs along at least three largely independent axes:

- **Generalization:** transductive (fixed graph, e.g. GCN as originally proposed) vs. inductive (generalizes to unseen nodes/graphs, e.g. GraphSAGE, GIN, and GAT when trained that way).
- **Weighting mechanism:** structure-fixed (GCN's degree normalization), sampled/uniform (GraphSAGE's mean/pool over a sample), or learned and content-dependent (GAT's attention).
- **Expressiveness:** whether the aggregation is injective enough to reach WL-test-level power (GIN), or provably weaker (GCN, GraphSAGE with mean/max, GAT in the worst case — see Lesson 10).

The important, slightly counterintuitive point from Lesson 10: expressiveness and content-adaptive weighting are *different* axes. GAT's attention makes weighting smarter, but an attention-weighted sum still isn't proven injective — it doesn't automatically buy WL-level power. GIN reaches the top of the expressiveness axis with a comparatively simple, uniform sum, not with attention. Don't assume "more sophisticated weighting" implies "more expressive."

A decision flowchart

As a starting heuristic — not a strict rule, since real projects often benchmark more than one — the two questions that matter most are whether you need to generalize beyond the training graph, and what the task actually demands once you're in that branch:



A starting heuristic, not a strict rule. GAT can also be trained inductively (Lesson 9); GIN's sum-and-MLP idea can substitute for GraphSAGE's aggregator when scalability and fine expressiveness both matter.

Applying it to real scenarios

- **Node classification on a single citation network (e.g. Cora, Citeseer), full graph known at training time:** No new nodes expected, neighbors roughly interchangeable → **GCN** is the efficient default.
- **Recommender or social graph where you want to explain "why was this neighbor influential?":** Fixed or slowly-changing graph, but neighbor relevance varies sharply → **GAT**, whose attention weights double as an interpretability signal.

- **Fraud/AML transaction graph where new accounts and transactions appear continuously:** New nodes constantly, and you can't afford to retrain before scoring them → **GraphSAGE** (or GAT trained inductively) — this is precisely the "future AML application" framing from the mission.
- **Molecular property prediction where structural isomers must be told apart:** Graph-level task, every molecule is a separate example (inherently inductive), and fine structural distinctions matter → **GIN**, with its WL-level expressiveness.

Key idea: The four architectures aren't mutually exclusive design points — they're four corners of the same design space (weighting mechanism × generalization × expressiveness). Modern practice frequently mixes ideas: a GraphSAGE-style sampled, inductive pipeline using a GIN-style sum aggregator, or an attention mechanism restricted to sampled neighbors. Knowing *why* each original architecture made its choice is what lets you recombine them sensibly.

For the full side-by-side breakdown — aggregation type, weighting, transductive/inductive, expressiveness, typical use case, and key limitation for all four architectures in one table — see the Architecture Comparison Card

(../reference/architecture-comparison-card.html).

Check your understanding

1. A GNN must score fraud-detection accounts that didn't exist when the model was trained. Which property should the architecture have?

Inductive generalization to previously unseen nodes

Transductive generalization to previously unseen nodes

Inductive memorization of previously seen nodes

Transductive memorization of previously seen nodes

Scoring nodes absent at training time requires inductive generalization — parameters (aggregators, attention, MLPs) that are functions of local structure and features, not tied to one fixed graph's identity, the way GraphSAGE, GIN, and inductively-trained GAT are.

2. A molecule-property task needs to distinguish two structurally different isomers with identical degree sequences. Which architecture is most likely to succeed, given Lesson 10's expressiveness results?

GIN, because sum aggregation is injective

GCN, because mean aggregation is injective

GAT, because attention aggregation is injective

GraphSAGE, because pooling aggregation is injective

GIN's sum-based aggregation, combined with an MLP, is the only one of the four proven capable of injectively encoding neighbor multisets and thus reaching WL-test-level expressiveness — exactly what's needed to tell apart structurally distinct graphs that simpler aggregators would conflate.

3. Why doesn't GAT's attention mechanism guarantee WL-level expressiveness, even though it learns content-dependent weights?

Attention-weighted sum is not proven to be injective

Attention-weighted sum is not proven to be symmetric

Attention-weighted sum is not able to be computed

Attention-weighted sum is not able to be normalized

Expressiveness and content-adaptive weighting are separate axes. Learning better weights doesn't by itself guarantee that different neighbor multisets always map to different aggregated outputs — GIN's guarantee comes from injectivity of sum-plus-MLP, not from how the weights are chosen.

Primary source

Read: Wu et al., "A Comprehensive Survey on Graph Neural Networks"

(<https://arxiv.org/abs/1901.00596>) (2021), Sections 3–4, for the broader taxonomy (recurrent, convolutional/spectral, convolutional/spatial, attentional) that GCN, GraphSAGE, and GAT each instantiate, and how they're evaluated across benchmarks.

Questions about anything in this lesson? Ask your teacher — run `/teach` and ask about whatever is unclear. Next up: **Lesson 12 — Readout and Pooling for Graph-Level Tasks**, where we look at how per-node embeddings from any of these architectures get combined into a single graph-level prediction.

Readout and Pooling for Graph-Level Tasks

After L rounds of message passing, every node v holds an embedding $h_v^{(L)}$ that summarizes its L -hop receptive field. For node- and edge-level tasks, that's the whole story: feed h_v (or a pair h_u, h_v) straight into an output head. But a graph-level task — is this molecule toxic? will this transaction network be flagged? — needs exactly *one* vector per graph. Lesson 1 promised "we'll cover this in a later lesson." This is that lesson.

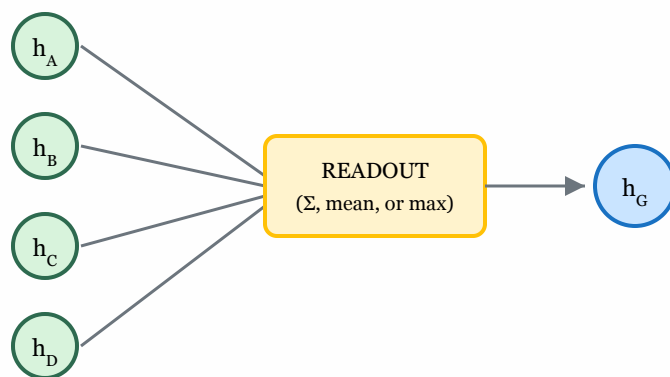
From a set of nodes to a graph

Readout (also called graph pooling) is a function that maps the set of final node embeddings $\{h_v : v \in V\}$ to a single vector h_G . It has one hard requirement, and it isn't new: h_G must be **permutation invariant**, not merely equivariant. A graph has no canonical node order (Lesson 1, Lesson 4), so relabeling the nodes and re-running readout must produce the identical h_G — not a permuted version of it, the *same* vector. Every option below is built to satisfy that constraint by construction: they all treat $\{h_v\}$ as an unordered multiset.

Simple readout: sum, mean, max

The three workhorse readouts, each applied dimension-by-dimension across all nodes in the graph:

- **Sum:** $h_G = \sum_{v \in V} h_v$
- **Mean:** $h_G = (1/|V|) \sum_{v \in V} h_v$
- **Max:** $h_G = \text{elementwise max}_{v \in V} h_v$



Four per-node embeddings collapse into one permutation-invariant graph embedding.

All three are cheap, differentiable, and add zero parameters. But they aren't interchangeable:

- **Sum** is sensitive to $|V|$: a graph with twice as many nodes of the same "type" produces roughly twice the magnitude. That's a feature, not a bug, when graph size itself is informative (a bigger molecule has more atoms to be toxic with).
- **Mean** normalizes size out. Useful when you care about the average composition of a graph regardless of how big it is, but it can wash out the signal from a small number of decisive nodes in a large graph.
- **Max** keeps only the single most extreme value per feature dimension across all nodes — good for "does this graph contain a motif with property X anywhere," bad if the task needs a holistic profile rather than a peak.

Key idea: The GIN paper (Lesson 10) makes this precise: under mild conditions, **sum** is the only one of the three that is *injective* over multisets of node embeddings — two different multisets are (almost) guaranteed to sum to different vectors, while mean and max can collide on genuinely different inputs. That's exactly why GIN pairs a sum-aggregator in its message-passing step with a sum-readout at the graph level: injectivity all the way through is what lets GIN match the Weisfeiler-Leman test's discriminative power (more on this in Lesson 15).

In practice, readout has to cooperate with batching (Lesson 2): a batch is one big disjoint-union graph plus a `batch` vector recording which original graph each node belongs to. Readout becomes a *scatter* operation — sum/mean/max each node's embedding into the slot given by its batch index, producing one h_G per graph in the batch in a single pass. This is exactly what PyTorch Geometric's `global_add_pool`, `global_mean_pool`, and `global_max_pool` do under the hood.

Where flat readout falls short

All three simple readouts treat the final-layer embeddings as an unstructured bag: every node contributes to h_G exactly once, exactly the same way, regardless of where it sits in the graph. But many real graphs have structure *above* the node level — a molecule is built from functional groups, a social network from communities, a fraud ring from tightly-connected clusters loosely linked to the rest of the account graph. A single flat pooling step at the very end has no way to represent "these fifteen nodes form one unit, and that unit relates to these other units."

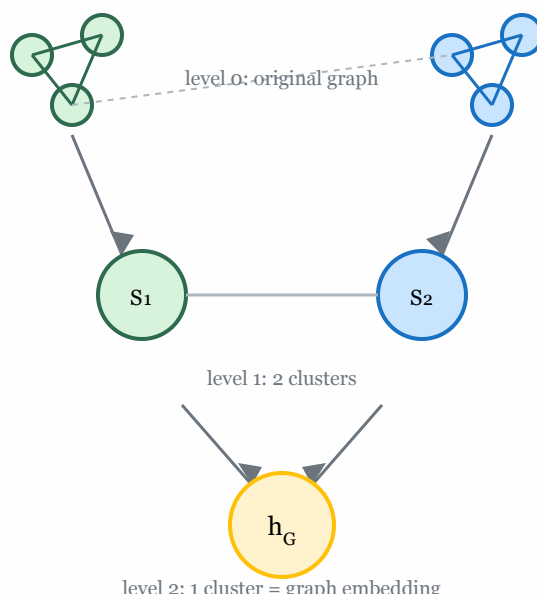
Hierarchical pooling: DiffPool

DiffPool (Ying, You, Morris, Ren, Hamilton & Leskovec, "Hierarchical Graph Representation Learning with Differentiable Pooling" (<https://arxiv.org/abs/1806.08804>), NeurIPS 2018) takes the CNN analogy from Lesson 1 further: instead of pooling once at the end, coarsen the graph progressively across several layers, the way a CNN shrinks spatial resolution with each pooling layer.

At each pooling layer l , two GNNs run in parallel over the current (already-coarsened) graph:

- One produces ordinary node embeddings $Z^{(l)}$, same as any message-passing layer.
- The other produces a **soft cluster-assignment matrix** $S^{(l)} \in \mathbb{R}^{n_l \times n_{l+1}}$ — each row softmax-normalized, so every node gets a distribution over the next layer's (smaller number of) clusters, rather than a hard 0/1 assignment.

The graph is then coarsened using both: $X^{(l+1)} = S^{(l)\top} Z^{(l)}$ (each new cluster's features are a weighted sum of its member nodes' embeddings), and $A^{(l+1)} = S^{(l)\top} A^{(l)} S^{(l)}$ (each new cluster-to-cluster edge weight is aggregated from the edges between their members). Repeat for a handful of layers until only one cluster remains — that final vector is h_G , no separate readout call needed.



DiffPool coarsens the graph across layers instead of pooling flat in one step. Learned soft assignments (S_1, S_2) decide which nodes merge into which cluster.

Key idea: DiffPool is trained *end-to-end* with the rest of the network via ordinary backprop — the cluster assignments are learned jointly with everything else, not computed by a separate graph-clustering algorithm (e.g. spectral clustering) run as a preprocessing step. In practice it's trained with two auxiliary terms: a link-prediction-style reconstruction loss (encouraging $S^{(l)} S^{(l)\top}$ to resemble the real adjacency $A^{(l)}$, so clusters respect actual connectivity) and an entropy regularizer (pushing each row of $S^{(l)}$ toward one-hot, so assignments stay interpretable rather than diffuse).

Analogy: DiffPool's learned cluster assignment does for graphs what a strided pooling layer does for a CNN feature map — shrinking the representation while keeping locally-salient structure. The difference is that a CNN's pooling

windows are handed to it for free by the fixed grid; DiffPool has to *learn* which nodes belong together, because graphs don't come with a fixed neighborhood shape to slide a window over.

DiffPool isn't the only hierarchical option — lighter-weight alternatives like top-k / SAGPool score each node with a learned function and simply drop the lowest-scoring fraction, avoiding the dense $S^{(l)}S^{(l)\top}$ computation. The soft-assignment idea above is the one worth having internalized, though: it's the "next level up" from flat readout that the rest of the literature riffs on.

Check your understanding

1. GIN's authors argue that one readout function is best able to distinguish graphs that differ only in node count (multiset cardinality), because it isn't normalized by node count. Which one?

Sum pooling

Mean pooling

Max pooling

Softmax pooling

Sum scales with $|V|$, so it's sensitive to node count. Mean explicitly normalizes count out, and max keeps only one extreme value regardless of how many nodes produced it — both lose count information that sum preserves.

2. What problem does DiffPool address that flat sum/mean/max readout does not?

It captures the graph's community structure

It captures the graph's node features

It captures the graph's edge weights

It captures the graph's degree sequence

Flat readout already operates on node embeddings (which encode features and, indirectly, local structure) — what it can't represent is that groups of nodes form higher-level units. DiffPool learns soft cluster assignments across several coarsening layers to capture exactly that hierarchical/community structure.

3. During graph batching, what tells the readout step which nodes belong to which original graph?

The node feature matrix

The edge index tensor

The batch index vector

The dense adjacency matrix

Batching (Lesson 2) disjoint-unions the graphs and keeps a separate batch vector recording each node's source graph. Readout uses that vector to scatter-sum/mean/max nodes into the correct per-graph slot, since the node and edge tensors alone don't carry graph membership.

Primary source

Read: "Graph Representation Learning" (https://www.cs.mcgill.ca/~wlh/grl_book/) by William L. Hamilton, **Chapter 6**, for the formal treatment of graph-level readouts and pooling. Then: "Hierarchical Graph Representation Learning with Differentiable Pooling"

(<https://arxiv.org/abs/1806.08804>) by Ying, You, Morris, Ren, Hamilton & Leskovec (NeurIPS 2018) for DiffPool itself — the coarsening figures in the paper are worth studying directly.

Questions about anything in this lesson? Ask your teacher — run `/teach` and ask about whatever is unclear. Next up: **Lesson 13 — Loss Functions and Training Strategies**, where we'll look at the graph-specific gotchas in the training pipeline itself: transductive splits, edge leakage, and negative sampling.

← Lesson 11: Architecture Comparison Lesson 13: Loss Functions and Training Strategies →

Loss Functions and Training Strategies

Lessons 4–11 built the encoder: message passing, receptive fields, and the GCN/GraphSAGE/GAT/GIN family. Lesson 12 finished it off with readout for graph-level outputs. The encoder is fully specified — but wrapping a training loop around it is where an ML-experienced practitioner is most likely to get tripped up, because graphs quietly break the i.i.d. assumption almost every standard training pipeline is built on.

The losses themselves are familiar

Nothing new here, so we'll be brief: node classification uses per-node cross-entropy (masked to whichever nodes have labels); graph classification uses cross-entropy on the pooled h_G from Lesson 12; regression tasks use MSE at whichever level applies. Optimizers, learning rate schedules, regularization — all the same tools from your CNN/RNN experience transfer directly. The graph-specific work is entirely in *what data reaches the loss*, not in the loss function's form.

Transductive training: the same graph, in and out

Recall the transductive-vs-inductive distinction that's been threaded through the architecture lessons. In a **transductive** node-classification setup — the classic Cora / Citeseer / Pubmed benchmarks the original GCN paper reports on — there is one fixed graph, and the train, validation, and test node sets are all subsets of its nodes. Critically, *all* of them are present in the graph during training: their features and their edges are visible to every message-passing step, every epoch. Only their **labels** are hidden from the loss via a mask.

This is a fundamentally different shape from an i.i.d. tabular or image split, where test rows are physically absent from the training data entirely. A GNN training on a transductive split is, on every forward pass, aggregating information that flows

through test nodes' features and edges — it just never gets gradient signal from their labels.

Key idea: "Transductive" does not mean "no leakage risk." It means the leakage boundary has moved from *features and structure* (safely shared) to *labels only* (must stay masked). Masking labels correctly is necessary but not automatically sufficient — the next section shows a case where structure itself has to be masked too.

This also isn't just an evaluation-protocol footnote for your AML/fraud interest. A fraud graph gains new accounts and transactions constantly; the transductive assumption — that the full node set is fixed and known ahead of time — breaks on day one of deployment. That's the practical reason GraphSAGE-style inductive aggregation (Lesson 8) matters operationally, not just as an architecture-comparison talking point.

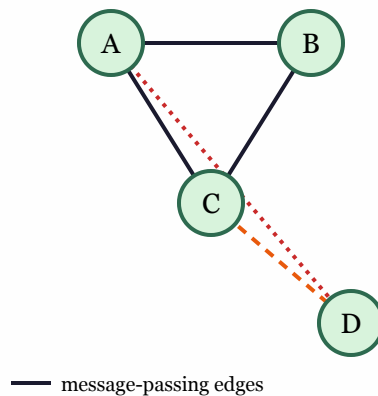
Splitting edges for link prediction without leaking the answer

Link prediction has a subtlety node classification doesn't: the thing being predicted — "does an edge exist between u and v ?" — is also part of the input that message passing consumes. If you train on the full adjacency matrix and then ask the model to score a training edge, a 1-layer GNN can simply "see" that edge directly in u and v 's own neighborhoods and answer correctly for the wrong reason. The model looks like it's learned to predict links; it has actually learned to look up an entry it was already given.

The fix is to partition edges into disjoint roles *before* anything else happens:

- **Message-passing edges:** the adjacency the encoder actually convolves over to produce embeddings.
- **Supervision edges:** the positive examples the loss trains on — drawn from edges deliberately excluded from the message-passing set above.

- **Held-out validation/test edges:** excluded from training-time message passing *and* from the training loss entirely.



Solid black = message-passing edges (A–B, A–C, B–C). Dashed orange (C–D) = supervision edge, held out of message passing but used in the loss. Dotted red (A–D) = test edge, held out of everything at training time.

Key idea: The edge you're training the model to predict must never be one it can also convolve over in that same training step. Libraries like PyTorch Geometric bake this discipline in — its `RandomLinkSplit` utility produces exactly these disjoint message-passing / supervision / test edge sets rather than a single naive edge shuffle.

Negative sampling

A graph only records positive facts: an edge that exists. To turn "does this edge exist?" into a trainable classification or ranking loss, you need negative examples too — node pairs with no edge between them — and the graph doesn't hand those to you directly.

Negative sampling manufactures them: for each positive edge (u, v) , sample one or more node pairs that are *not* edges — commonly by fixing u and drawing a random v' not in u 's neighbor set. The ratio of negatives to positives per positive edge is a tunable hyperparameter, same as it would be for any imbalanced binary classification setup. Two loss shapes are standard once you have them: binary

cross-entropy on $\text{sigmoid}(\text{score}(u,v))$ treating positive/negative as classes, or a margin/ranking loss such as Bayesian Personalized Ranking (BPR), which only asks that $\text{score}(\text{positive})$ exceed $\text{score}(\text{negative})$ by some margin rather than pinning down an absolute probability.

One detail worth internalizing: negatives should be *resampled every training epoch* (so the model can't just memorize one fixed negative set), but *fixed once* for validation and test (so the reported metric is comparable run to run, rather than drifting because the negative pairs changed).

Self-supervised objectives when labels are scarce

Labeled nodes or edges are frequently a tiny fraction of a real graph — a handful of confirmed fraud accounts out of millions, say. Self-supervised pretraining lets you exploit the graph's full structure and features with no labels at all, then fine-tune (or just train a lightweight classifier on top of frozen embeddings) using the scarce labeled subset.

Link prediction itself, described above, is one such self-supervised pretext task — it needs no labels beyond the graph's own edges. A more general contrastive alternative is **Deep Graph Infomax** (DGI; Veličković et al., arXiv:1809.10341 (<https://arxiv.org/abs/1809.10341>), ICLR 2019): it trains the encoder to maximize agreement between each node's local embedding and a global graph summary vector (computed via the same kind of readout from Lesson 12), while minimizing agreement with a summary computed from a *corrupted* version of the graph (e.g. node features randomly shuffled across nodes) used as the negative signal. No labels enter the loss at all — only the graph's own structure and features, contrasted against a deliberately-broken version of itself.

Analogy: DGI is the graph analogue of contrastive pretraining in vision (SimCLR-style: pull together representations of two augmented views of the same image, push apart representations of different images). Swap "augmented view" for "corrupted graph," and "two images" for "a node's local

embedding vs. the whole graph's summary," and it's the same recipe: agree with your true context, disagree with a shuffled decoy.

DeepWalk and node2vec (next lesson) are the historical predecessors of this idea — self-supervised objectives that also need no labels, just predating the contrastive-with-a-GNN-encoder framing DGI uses.

Check your understanding

1. In a transductive node-classification setup on a single citation graph, what is actually hidden from the model during training?

Only the test node labels

Only the test node features

Only the test node edges

The entire test subgraph structure

Transductive training keeps the full graph — features and edges of every node, including test nodes — visible during every forward pass. A mask hides only the test labels from the loss, which is a much narrower boundary than an i.i.d. split where test rows are absent entirely.

2. If test edges remain in the message-passing graph during link-prediction training, what happens?

The model directly observes answers

The model trains far slower

The model ignores all edges

The model cannot use features

A GNN convolving over an edge it's also being asked to predict can simply "see" that edge in its own receptive field and answer correctly without learning to generalize. That's why message-passing, supervision, and held-out edges must be kept in disjoint sets.

3. Why is negative sampling necessary when training a link-prediction model?

Graphs only contain positive edge examples

Graphs only contain negative edge examples

Positive edges are always mislabeled data

Negative edges speed up gradient computation

A graph records only edges that exist — there is no built-in list of "non-edges." Negative sampling manufactures node pairs with no edge so the loss has something to contrast the true edges against, turning link prediction into a trainable binary classification or ranking problem.

Primary source

Watch/read: Stanford CS224W (<https://cs224w.stanford.edu/>), **Lecture 5** ("A General Perspective on GNNs" / training GNNs), for the full treatment of prediction heads, splits, and loss design. Pair with Hamilton's "Graph Representation Learning," Chapter 6 (https://www.cs.mcgill.ca/~wlh/grl_book/) for the formal training-and-evaluation framing, and Veličković et al., "Deep Graph Infomax" (<https://arxiv.org/abs/1809.10341>) for the self-supervised objective discussed above.

Questions about anything in this lesson? Ask your teacher — run `/teach` and ask about whatever is unclear. Next up: **Lesson 14 — Node Embeddings Before GNNs**, where we'll look at DeepWalk, node2vec, and LINE: the shallow, skip-gram-based methods that predate message passing.

← Lesson 12: Readout and Pooling for Graph-Level Tasks Lesson 14: Node Embeddings Before GNNs →

Node Embeddings Before GNNs

Lessons 4–13 built node embeddings the message-passing way: gather, aggregate, update, stack layers, pool, train. But that isn't how the field got its first working node embeddings. From roughly 2014 to 2016 — before Kipf & Welling's GCN paper (2017) made message passing the default — the dominant approach never looked at a neighbor's features at all. This lesson covers that predecessor family: DeepWalk, node2vec, and LINE.

The shallow embedding recipe

Compare two ways of assigning node v a vector:

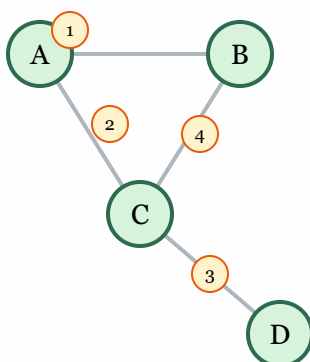
- **GNN encoder** (Lesson 4 onward): $z_v = f(\text{features of } v \text{ and its neighbors; shared weights})$ — a function, applied to local structure and features.
- **Shallow embedding**: $z_v = Z[v]$ — a single row of a directly-learned lookup table $Z \in \mathbb{R}^{|V| \times d}$, indexed by node ID. No features, no neighbor aggregation, no shared function — just one free vector per node, optimized directly.

The training objective is borrowed wholesale from NLP: **skip-gram**, the same idea behind word2vec (Mikolov et al., 2013). Word2vec treats "words that co-occur in a sentence window" as a training signal — nearby words should have similar embeddings. Shallow graph embeddings treat "nodes that co-occur in some generated sequence" the same way. The only real design question separating DeepWalk, node2vec, and LINE is: what counts as a sequence, and what counts as "nearby"?

DeepWalk: random walks as sentences

DeepWalk (Perozzi, Al-Rfou & Skiena, arXiv:1403.6652 (<https://arxiv.org/abs/1403.6652>), KDD 2014) generates many **random walks** from every node: starting at v ,

repeatedly hop to a uniformly-random neighbor for some fixed number of steps. Each walk is a sequence of node IDs — treat it exactly like a sentence of words, and run skip-gram (with negative sampling) over it: slide a context window along the walk, and train Z so that a center node's embedding predicts the nodes that appear in its window.



A random walk $A \rightarrow C \rightarrow D \rightarrow C \rightarrow B$ (steps 1–4) produces the "sentence" $[A, C, D, C, B]$ that skip-gram trains on.

Nodes that co-occur often across many such walks — because they're densely interconnected or otherwise structurally close — end up with similar embeddings. That's the whole mechanism: no features, no explicit aggregation function, just "predict my random-walk neighbors."

node2vec: biasing the walk

node2vec (Grover & Leskovec, arXiv:1607.00653 (<https://arxiv.org/abs/1607.00653>), KDD 2016) keeps DeepWalk's skip-gram-over-walks skeleton but generalizes the walk itself from uniform to a **2nd-order biased random walk**, controlled by two hyperparameters: return parameter p (how likely the walk is to immediately backtrack to the node it just came from) and in-out parameter q (how likely it is to venture further away vs. stay local). Formally, having just moved from t to the current node v , the unnormalized weight for stepping to a candidate x is scaled by $1/p$ if $x = t$ (returning), 1 if x is also a neighbor of t (staying local), and $1/q$ if x is farther from t than v is (exploring outward).

Key idea: Low q biases the walk toward DFS-like exploration, producing embeddings that capture *homophily* — nodes in the same community end up close in embedding space. High q biases toward BFS-like exploration, capturing *structural equivalence* — nodes that play the same structural role (e.g. "hub" or "bridge") end up close even if they sit in entirely different parts of the graph. p and q only change the sampling strategy; the embedding table and skip-gram loss underneath are otherwise identical to DeepWalk.

LINE: no walks at all

LINE (Tang et al., arXiv:1503.03578 (<https://arxiv.org/abs/1503.03578>), WWW 2015) drops random walks entirely and optimizes two proximities directly: **first-order proximity** (nodes joined by an edge should have similar embeddings — a direct pairwise signal) and **second-order proximity** (nodes with similar neighborhoods, even without a direct edge, should have similar embeddings — the "shared context" idea from skip-gram, computed from shared neighbors instead of walk co-occurrence). Each proximity is trained as its own embedding set, then the two are concatenated. LINE's motivation was explicit: scale to production social networks with hundreds of millions of edges, where generating and storing large numbers of random walks is itself a cost worth avoiding.

Why GNNs superseded shallow embeddings for most uses

Three structural limitations, all stemming from the same root cause — Z is a table indexed by node identity, not a function of anything:

1. **No node features used.** DeepWalk/node2vec/LINE see only graph topology and node ID; any real feature vector you have — bag-of-words on a paper, transaction statistics on an account — is simply discarded. A GNN encoder consumes those features directly at every message-passing step.

2. **Fixed to the training graph — no inductive generalization.** Z's rows correspond one-to-one to the node IDs present when training ran. A node that shows up afterward has no row and no way to get one short of rerunning the whole skip-gram training procedure. GraphSAGE, GAT, and GIN instead learn a function of local neighborhood features that applies to any node, seen or not, at inference time (Lesson 8's inductive framing). This is precisely the everyday reality of a fraud graph: new accounts and transactions appear continuously, so a detection pipeline that needs a full embedding-table retrain before it can score a new account is a non-starter operationally.
3. **No parameter sharing across the graph.** Z has $O(|V| \times d)$ parameters — growing linearly with the graph — and each node's vector is optimized almost independently. A GNN's weight matrices are shared across every node and scale with feature dimension rather than node count, which is both a smaller model and, through that shared function, an implicit way for nodes to borrow statistical strength from each other.

Where shallow embeddings still make sense

None of this makes DeepWalk/node2vec/LINE obsolete everywhere:

- **Featureless, purely structural graphs** — where topology genuinely is all you have, so a GNN's feature-consuming machinery has nothing extra to exploit.
- **Fixed, closed-world, transductive graphs** — the node set is known upfront and won't grow, so the inductive-generalization limitation never bites.
- **Cheap baselines and fast prototyping** — skip-gram training is lightweight compared to backpropagating through variable-depth neighborhood aggregation, and it's often the fastest way to get a first embedding to sanity-check against.
- **Coarse structural-similarity tasks** — visualization, quick clustering — where a discriminative signal tied to rich features isn't the point.

Key idea: Shallow embeddings and GNN encoders aren't mutually exclusive. It's common to see pretrained DeepWalk or node2vec vectors used to initialize a GNN's input features, or included as one input feature among several — the historical predecessor becomes a component of the modern pipeline rather than a competitor to it.

Check your understanding

1. What is the main input to a DeepWalk or node2vec embedding, unlike a GNN?

Only the node's identity

Only the node's features

Only the node's degree

Only the node's neighbors

Shallow embeddings look up a row of a table indexed purely by node ID. They never consume node features, degree, or neighbor information directly — any structural signal they capture comes indirectly, through which nodes co-occur in random walks.

2. In node2vec, what do the return parameter p and in-out parameter q jointly control?

How biased the random walk is

How large the embedding table is

How many negative samples are used

How many GNN layers are stacked

p and q bias the 2nd-order random walk toward backtracking, staying local, or exploring outward — interpolating between DFS-like walks (homophily) and BFS-like walks (structural equivalence). They don't change the table size, the negative-sampling count, or anything about GNN layers — node2vec has no message-passing layers at all.

3. A fraud-detection graph gains thousands of new accounts every day. Why is this a problem for a DeepWalk-style shallow embedding?

New nodes have no learned embedding row

New nodes always get duplicate embedding rows

New edges always overwrite existing embedding rows

New features always corrupt existing embedding rows

Z is indexed by the node IDs seen during training; a node that didn't exist then has no row in the table and can't get one without rerunning training. An inductive GNN encoder, by contrast, computes an embedding as a function of local neighborhood features that works on nodes it has never seen.

Primary source

Read: "Graph Representation Learning" (https://www.cs.mcgill.ca/~wlh/grl_book/) by William L. Hamilton, **Chapters 3–4**, for the systematic treatment of shallow embeddings and the encoder-decoder framing that unifies DeepWalk, node2vec, and LINE. Then: Grover & Leskovec, "node2vec: Scalable Feature Learning for Networks" (<https://arxiv.org/abs/1607.00653>) (2016) for the biased random walk itself.

Questions about anything in this lesson? Ask your teacher — run `/teach` and ask about whatever is unclear. Next up: **Lesson 15 — The Weisfeiler-Leman Hierarchy**, where we'll pin down exactly what GNNs can and can't distinguish between graphs.

← Lesson 13: Loss Functions and Training Strategies

Lesson 15: The Weisfeiler-Leman Hierarchy →

The Weisfeiler-Leman Hierarchy

Back in Lesson 10, GIN was motivated by a claim: it's as powerful as the Weisfeiler-Leman (WL) test at telling graphs apart. We took that on faith to get to the architecture. This lesson cashes out the claim — what the WL test actually is, why message passing turns out to be doing the same computation, and what "as powerful as 1-WL" implies about the hard ceiling on *any* standard GNN's ability to distinguish graphs, no matter how it's trained or how many layers it has.

Expressiveness is about telling graphs apart

A GNN is, structurally, a function that maps a graph to a set of embeddings. Call two graphs **indistinguishable** by a GNN if it maps them (or corresponding nodes in them) to identical embeddings, for every possible choice of weights.

"Expressiveness" is a question about the function class: which pairs of non-isomorphic graphs *can* some GNN in this class map to different embeddings, and which pairs can no GNN in the class ever separate, regardless of training?

This is a statement about representational capacity, not about what a specific trained model happens to learn — the same flavor of question as "can an MLP with ReLUs represent XOR," just for graph-structured inputs.

Key idea: Expressiveness ceilings are architecture properties, not training artifacts. If two non-isomorphic graphs are indistinguishable to the GNN's function class, no amount of data or optimization will ever separate them — the information was destroyed by the architecture itself.

The 1-WL color-refinement algorithm

The Weisfeiler-Leman test (1-dimensional version, also called **color refinement**) is a classical, cheap heuristic for graph isomorphism. In outline:

1. **Initialize** every node's "color" from its label or degree.
2. **Refine:** repeatedly recolor each node by hashing its current color together with the *multiset* of its neighbors' colors.
3. **Stop** when the coloring stops changing (the partition is stable).
4. **Compare:** two graphs are declared possibly-isomorphic if their final color histograms match; if the histograms differ, the graphs are *definitely* non-isomorphic.

The full step-by-step mechanics, with two worked examples, live in the WL Test Reference ([../reference/wl-test-reference.html](http://reference/wl-test-reference.html)) — read it alongside this lesson if the algorithm above feels abstract.

Message passing is color refinement, learned

Look again at the gather → aggregate → update loop from Lesson 4 and set it side by side with the steps above:

1-WL color refinement	Message-passing GNN
Initial color from node label/degree	Initial hidden state $h_v^{(0)}$ from node features
Gather multiset of neighbor colors	Gather multiset of neighbor hidden states (AGGREGATE)
Hash(own color, neighbor multiset)	Learned COMBINE/UPDATE function
Compare final color histograms	Pooled/readout graph embedding (Lesson 12)

They're the same algorithm skeleton. The WL hash function is a fixed, injective (but otherwise arbitrary) recoloring rule; a GNN replaces it with a *learned* AGGREGATE + COMBINE. Xu et al. (2019) make this precise: for any message-passing GNN, if the GNN maps two graphs to different embeddings, then 1-WL also distinguishes them. In other words, **a message-passing GNN's discriminative power is**

bounded above by 1-WL — no message-passing architecture, however deep or wide, can ever tell apart two graphs that 1-WL says are indistinguishable.

Key idea: 1-WL isn't just an analogy for message passing — it's an upper bound on what message passing can ever compute, provably, for the entire architecture family (GCN, GraphSAGE, GAT, GIN, and any future variant built on gather-aggregate-update).

GIN reaches the ceiling

The bound above says GNNs are *at most* as powerful as 1-WL. Whether a specific GNN actually *reaches* that ceiling depends on whether its AGGREGATE and COMBINE steps are injective over multisets of neighbor features — if two different neighbor multisets ever get squashed to the same aggregated value, the GNN has thrown away information 1-WL would have kept.

This is exactly the design problem GIN (Lesson 10) solves. Xu et al. show that sum aggregation, followed by an MLP, can represent an injective function over any countable multiset domain — the "deep multisets" result. Mean and max aggregation cannot: mean conflates a neighbor multiset $\{1, 1\}$ with $\{1\}$ rescaled, and max conflates $\{1\}$ with $\{1, 1, 1\}$, throwing away exactly the multiplicity information 1-WL's hash preserves. That's why GIN's $(1+\epsilon) \cdot h_v + \sum_{u \in N(v)} h_u$ update, feeding into an MLP, is provably as powerful as 1-WL — it hits the ceiling that GCN, GraphSAGE, and GAT (with their mean/attention-weighted aggregations) provably fall short of.

Analogy: This mirrors why sum-pooling outperforms average-pooling for graph-level readout when you need to preserve count information (Lesson 12) — the same injectivity argument shows up twice, once inside each layer's aggregation and once in the final readout.

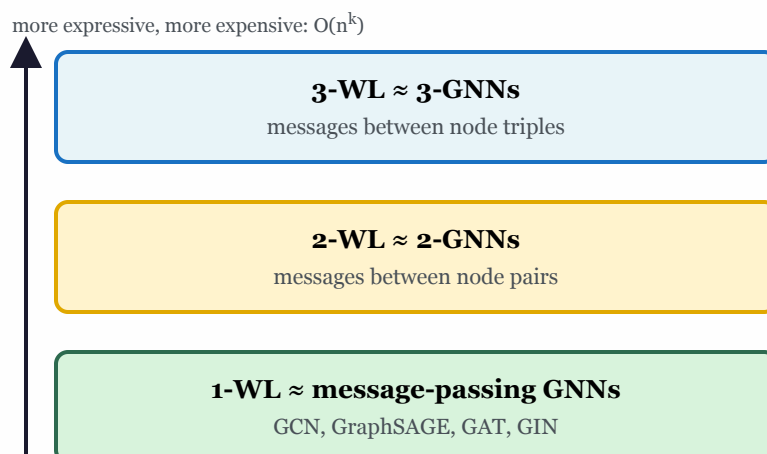
What 1-WL still can't see

Because GIN caps out at 1-WL, GIN inherits 1-WL's blind spots. The classic failure: any two **regular graphs** (every node has the same degree) with the same size can look identical to 1-WL, even when they're structurally very different — a 6-node cycle and two disjoint triangles are the standard example, worked through in the reference doc. If your task depends on distinguishing structures like that (counting triangles, detecting specific substructures, some molecular property prediction tasks), no amount of GIN depth or width will help; the information is gone at the aggregation step, not lost to undertraining.

The k-WL hierarchy and higher-order GNNs

1-WL is the bottom rung of an infinite hierarchy. **k-WL** tests generalize color refinement to color *k-tuples* of nodes instead of single nodes, refining based on how each tuple's neighborhood of tuples looks. Each step up is strictly more powerful: 2-WL can separate some pairs 1-WL cannot (including the cycle-vs-triangles example above), 3-WL separates some pairs 2-WL cannot, and so on.

Morris et al. (2019, "Weisfeiler and Leman Go Neural") build the neural analogue: **k-GNNs**, which pass messages between k-tuples of nodes rather than single nodes, and prove k-GNNs match k-WL's expressive power exactly — the same equivalence argument as $\text{GIN} \leftrightarrow \text{1-WL}$, one level up.



Each rung of the WL hierarchy has a matching GNN family with provably equal expressive power. Cost grows fast: k -GNNs scale roughly as $O(n^k)$, which is why almost every GNN used in practice sits on the bottom rung.

In practice, k -GNNs for $k \geq 3$ are rarely used — the $O(n^k)$ tuple bookkeeping is prohibitive for anything but small graphs. Knowing the hierarchy exists is still valuable for reading papers: when a paper claims to "go beyond 1-WL," it's implicitly claiming to sit somewhere on this ladder (higher-order tuples, subgraph counting, or added structural features), and you can ask what it costs.

Check your understanding

1. According to Xu et al.'s theorem, a standard message-passing GNN's ability to distinguish non-isomorphic graphs is bounded above by which test?

The 1-WL test

The 2-WL test

The 3-WL test

The k -WL test

Xu et al. prove that any message-passing GNN can distinguish two graphs only if 1-WL also distinguishes them — 1-WL is the ceiling for the whole gather-aggregate-update architecture family, not the k -WL hierarchy in general.

2. Why does GIN use sum aggregation instead of mean or max over neighbor features?

It preserves injectivity over neighbor multisets

It preserves smoothness over neighbor multisets

It preserves symmetry over neighbor multisets

It preserves sparsity over neighbor multisets

Sum, followed by an MLP, can represent an injective function over any countable multiset of neighbor features. Mean and max both collapse multisets that differ only in multiplicity (e.g. $\{1,1\}$ vs $\{1\}$), throwing away information 1-WL's hash would have kept — which is why they fall short of the 1-WL ceiling.

3. 1-WL fails to distinguish which classic pair of non-isomorphic graphs?

A hexagon and two triangles

A hexagon and one triangle

A pentagon and two triangles

A hexagon and two squares

A 6-node cycle and two disjoint triangles are both 2-regular graphs (every node has degree 2), so every node looks identical under color refinement and the final histograms match — even though the graphs are structurally very different. See the WL Test Reference for the full worked example.

Primary source

Read: "How Powerful are Graph Neural Networks?" (<https://arxiv.org/abs/1810.00826>) by Xu et al. (2019) — the GIN paper, this time for its theoretical sections (the WL-bound theorem and the deep-multisets injectivity argument), not just the architecture. For the

hierarchy above 1-WL, see "Weisfeiler and Leman Go Neural: Higher-order Graph Neural Networks" (<https://arxiv.org/abs/1810.02244>) by Morris et al. (2019). And for the algorithm mechanics themselves, the WL Test Reference ([../reference/wl-test-reference.html](https://reference.wl-test-reference.html)) built alongside this lesson.

Questions about anything in this lesson? Ask your teacher — run `/teach` and ask about whatever is unclear. Next up: **Lesson 16 — Geometric Deep Learning: The Symmetry Perspective**, where the permutation-equivariance idea from Lesson 1 turns out to be one instance of a much bigger principle spanning CNNs, GNNs, and Transformers.

← Lesson 14: Node Embeddings Before GNNs

Lesson 16: Geometric Deep Learning →

Geometric Deep Learning: The Symmetry Perspective

Lesson 1 introduced permutation equivariance as GNNs' defining property, and drew a quick analogy: a CNN is a GNN on a grid graph with weight-sharing across the kernel; an RNN is a GNN on a chain graph with weight-sharing across time steps. This lesson takes that analogy and makes it load-bearing. It's not a loose metaphor — it's one instance of a single unifying principle that Bronstein, Bruna, Cohen, and Veličković formalize as the "geometric deep learning blueprint," and it's the most direct way to place any new architecture — GNN or otherwise — on a map.

The blueprint: domain, symmetry, invariance

Every architecture in the blueprint is described by the same three ingredients:

- **Domain.** The structure the data lives on — a grid, a graph, a manifold.
- **Symmetry group.** The set of transformations of that domain that shouldn't change what the network computes — translations of a grid, permutations of a graph's nodes, rotations of a sphere.
- **Invariance / equivariance.** The network's layers are built to respect that symmetry: shift the input according to the group, and the output either stays the same (invariance) or shifts the same way (equivariance) — exactly the node-level vs. graph-level distinction from Lesson 1's glossary terms, generalized to any domain.

Two further biases recur across the blueprint's architectures: **locality** (a layer only looks at a small neighborhood in the domain before combining wider context across layers) and **scale separation** (fine-grained local patterns can be composed into coarser ones by stacking layers or pooling). Both should sound familiar — they're exactly what stacking message-passing layers and pooling do in a GNN (Lessons 5 and 12).

Key idea: An architecture family isn't defined by "what layers it stacks" — it's defined by which symmetry group it's built to respect. Once you know the domain and the group, the rest of the architecture (weight sharing, locality, pooling) follows close to automatically.

The five geometries

The blueprint organizes deep learning's major architecture families by exactly this domain/symmetry pairing — sometimes called the "5 Gs":

Domain	Symmetry group	Architecture
Grids	Translation	CNNs
Groups	Rotation, reflection, general Lie/discrete groups	Group-equivariant networks
Graphs (& sets)	Permutation	GNNs, DeepSets
Geodesics & manifolds	Isometry (distance-preserving deformation)	Mesh / manifold networks
Gauges	Gauge symmetry (local frame choice)	Gauge-equivariant networks

The grids row is the one you already know cold: a convolution kernel is the same weights applied at every pixel position, which is exactly what makes a CNN invariant-equivariant to translation. The graphs row is the one this course has spent fourteen lessons on: message passing shares the same AGGREGATE/COMBINE function across every node, which is exactly what makes a GNN equivariant to node permutation. Same mechanism — weight sharing dictated by a symmetry group — different group.

Analogy: Grids and graphs sit at opposite ends of a structure spectrum. A grid gives every node the same fixed set of neighbor *offsets* (up, down, left, right), so the shared kernel can key its weights on position-in-the-neighborhood ("the weight for the pixel above me"). A graph gives no such canonical ordering of neighbors at all, so the shared function has to be defined over an unordered multiset instead — which is precisely why permutation invariance of the aggregation step (Lesson 4) isn't a nice-to-have, it's forced by the symmetry group graphs are equivariant to.

Manifolds and gauges, briefly

The other two rows extend the same idea to continuous, curved domains — meshes, point clouds, spheres — that show up in molecular surfaces, 3D shape analysis, and physics-informed models. A mesh network is built to be equivariant under isometries (deforming a surface without stretching it), and gauge-equivariant networks additionally handle the fact that there's no single consistent way to define "up" or "right" at every point on a curved surface, so the network has to be invariant to the arbitrary local coordinate frame ("gauge") chosen at each point. Both are out of scope for this course's depth, but knowing they exist — and that they're graphs' natural neighbors in the same framework, not an unrelated topic — is the point of the map.

Where the Transformer fits

One more placement worth making explicit, since it sets up the next lesson: a Transformer's self-attention layer, applied to a set of tokens with no positional information, is permutation-equivariant over that set — the same symmetry group as a GNN. In fact it's the special case of a GNN operating on the *complete graph* (every token attends to every other token, i.e. every node is connected to every other node). Language and vision Transformers add back the sequence/grid structure

through positional encodings; Lesson 17 asks what the equivalent move looks like when the domain is an arbitrary graph rather than a sequence.

Check your understanding

1. In the geometric deep learning blueprint, what does the CNN's convolution kernel weight-sharing correspond to?

Invariance under the translation group

Invariance under the permutation group

Invariance under the rotation group

Invariance under the reflection group

A convolution kernel applies the same weights at every spatial position, which is exactly the weight-sharing scheme that makes a CNN equivariant to shifting (translating) the input image.

2. Which architecture family corresponds to the "graphs" domain in the geometric deep learning blueprint?

Graph neural networks

Convolutional neural networks

Recurrent neural networks

Group equivariant networks

Graphs carry no canonical node ordering, so their symmetry group is the permutation group — and GNNs are the architecture family built to be equivariant to exactly that group, via permutation-invariant aggregation.

3. How does the blueprint describe the relationship between CNNs, GNNs, and Transformers?

They share one symmetry framework

They share one training framework

They share one pooling framework

They share one pruning framework

The blueprint's claim isn't about a shared training recipe or shared pooling scheme — it's that all three architectures are instances of the same domain/symmetry-group/(in)variance construction, just with different groups (translation, permutation, and permutation-over-the-complete-graph, respectively).

Primary source

Read: "Geometric Deep Learning: Grids, Groups, Graphs, Geodesics, and Gauges" (<https://geometricdeeplearning.com/>) by Bronstein, Bruna, Cohen, and Veličković (2021). Dense but conceptually powerful — the proto-book and lecture slides on the site are more approachable than the full arXiv preprint as a first pass. Focus on the "Blueprint of Geometric Deep Learning" section for the domain/symmetry/invariance framing used in this lesson.

Questions about anything in this lesson? Ask your teacher — run `/teach` and ask about whatever is unclear. Next up: **Lesson 17 — Graph Transformers**, where we

take the "Transformer = GNN on the complete graph" placement from this lesson and ask what has to be added back to make that useful.

← Lesson 15: The Weisfeiler-Leman Hierarchy

Lesson 17: Graph Transformers →

Graph Transformers

Lesson 16 placed the Transformer on the geometric map as a GNN operating on the complete graph — every token attends to every other token, with no structural bias at all. That placement raises the obvious question: if a Transformer already generalizes over an arbitrary set with full attention, why not just run it directly on a graph's nodes and let attention learn whatever structure matters? This lesson works through why that naive move fails, what has to be added back, and how the resulting **graph transformers** trade off against the message-passing GNNs you already know.

What plain self-attention throws away

Feed a graph's node features into a standard Transformer layer with no further changes, and every node attends to every other node with weights computed purely from feature similarity — the edge set `edge_index` never enters the computation. The result is provably weaker than message passing, not just structurally naive: a self-attention layer over a bag of node features, with no positional or structural information attached, is a permutation-equivariant function of the node *set* alone. It cannot tell a densely-connected graph from an empty one if the node features are the same, because it has no mechanism to look at the edges at all. Formally, "naked" graph transformers of this kind collapse to the same expressive class as DeepSets-style set functions — strictly weaker than 1-WL, and therefore weaker than even a single GCN layer.

Key idea: Full attention is permutation-equivariant over a *set*, but a graph is a set plus a structure. Dropping the structure isn't a simplification — it's a strictly weaker model than message passing, until something puts the edges back in.

Putting structure back: positional and structural encodings

The fix mirrors how vision and language Transformers handle their own domains: attach extra features to each node that encode where it sits in the graph, then let attention use them. Two schemes dominate:

- **Laplacian eigenvector positional encoding.** Compute the graph Laplacian (built from the adjacency and degree matrices) and take its lowest few eigenvectors as extra coordinates for each node. These generalize the sinusoidal positional encodings of sequence Transformers — on a chain graph, the Laplacian's eigenvectors literally *are* sinusoids — and capture *global* structural position: nodes in different parts of the graph get different coordinates even if their local neighborhoods look identical.
- **Random-walk positional encoding (RWPE).** For each node, record the probability that a k -step random walk started at that node returns to itself, for a range of step counts. This is cheaper to compute than an eigendecomposition and captures more *local* structural role — how tightly clustered a node's neighborhood is — rather than global position.

With either encoding concatenated onto the node features, attention weights can now be a function of graph structure, not just raw feature similarity — the encodings are what let a node "know" the shape of the graph around it before attention ever runs.

The GraphGPS recipe: message passing + attention, per layer

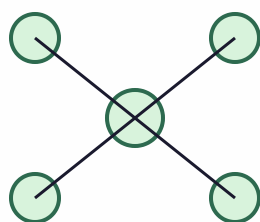
Rather than choosing between local message passing and global attention, Rampášek et al.'s GraphGPS framework (2022) runs both in every layer: a standard sparse message-passing block (any of GCN/GAT/GraphSAGE/GIN-style updates) alongside a global attention block, with their outputs combined before the next layer. Positional and structural encodings are computed once up front and fed to

both blocks. The result is a modular "recipe" rather than one fixed architecture — swap in whichever message-passing layer and whichever efficient attention variant suits the task.

Analogy: This is the same move as a Vision Transformer patchifying an image and adding positional embeddings before running full attention — ViT throws away the CNN's built-in locality prior and re-derives useful structure from data plus explicit position information instead. Graph transformers make the identical trade for graphs: give up hard-wired locality, buy it back (partially) with positional/structural encodings and, in hybrid designs like GraphGPS, a parallel message-passing path.

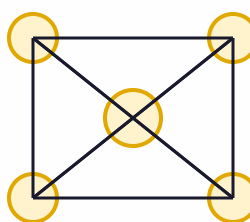
Trade-offs against message-passing GNNs

Message passing: sparse



$O(|E|)$ messages per layer

Full attention: dense



$O(n^2)$ attention per layer

Message passing only routes information along existing edges; full graph attention connects every node pair regardless of the original edges, recovering a global receptive field at quadratic cost.

Message-passing GNN

Receptive field grows one hop per layer (Lesson 5)

Prone to over-smoothing and over-squashing with depth (Lesson 6)

Graph transformer

Global receptive field from the first layer

Sidesteps both — distant nodes exchange information in one step, not through a narrow multi-hop bottleneck

Message-passing GNN	Graph transformer
$O(E)$ cost per layer — scales with sparsity	$O(n^2)$ cost per layer for full attention; needs sparsification or linear-attention tricks to scale
Locality is a hard architectural bias (only real edges matter)	Locality is a soft hint from positional/structural encodings, not enforced

"Over-squashing" here is the same bottleneck problem raised in Lesson 6's over-smoothing discussion, seen from a different angle: as a message-passing GNN gets deeper, information from a distant node has to travel through an exponentially growing set of paths and gets compressed into fixed-size vectors along the way, distorting long-range signals. A graph transformer's global attention doesn't need depth to reach far-away nodes, so it doesn't inherit that particular failure mode — at the price of losing the hard locality bias that made sparse message passing cheap and, for many tasks, a genuinely useful inductive bias rather than a limitation to route around.

Key idea: Graph transformers don't strictly dominate message-passing GNNs. They swap one set of failure modes (over-smoothing, over-squashing, limited receptive field) for another (quadratic cost, weaker locality bias, dependence on the quality of the positional/structural encoding). Which one wins is task- and graph-size-dependent — exactly the kind of judgment call the architecture-comparison framework from Lesson 11 should now extend to cover.

Check your understanding

1. Without any positional or structural encoding, what does a Transformer's self-attention over a node set effectively reduce to?

A permutation-equivariant function over sets

A permutation-equivariant function over grids

A permutation-equivariant function over paths

A permutation-equivariant function over trees

With no positional or structural information, attention weights depend only on feature similarity, not on the edge set. The resulting function is equivariant to permuting the node set but blind to graph structure — equivalent to a DeepSets-style set function, strictly weaker than 1-WL.

2. What is the main computational trade-off graph transformers make for a global receptive field from layer one?

Quadratic cost in node count

Quadratic cost in edge count

Quadratic cost in layer count

Quadratic cost in batch count

Full attention connects every pair of nodes, so a single layer costs $O(n^2)$ rather than the $O(|E|)$ of sparse message passing — the price for reaching every node in one step instead of one hop per layer.

3. Why can graph transformers avoid the over-squashing that troubles deep message-passing GNNs?

Every node attends to all nodes

Every node attends to one neighbor

Every node attends to its cluster

Every node attends to a subset

Over-squashing comes from information having to travel through an exponentially growing, narrowing set of multi-hop paths as depth increases. Full attention lets any two nodes exchange information directly in a single layer, so there's no long path for that information to be squashed through.

Primary source

Read: "Recipe for a General, Powerful, Scalable Graph Transformer"

(<https://arxiv.org/abs/2205.12454>) by Rampášek et al. (2022) — the GraphGPS paper, for the hybrid message-passing-plus-attention recipe and its treatment of positional/structural encodings. For the broader landscape of graph transformer designs, see Stanford CS224W Lecture 8 (<https://web.stanford.edu/class/cs224w/slides/o8-graph-transformer1.pdf>).

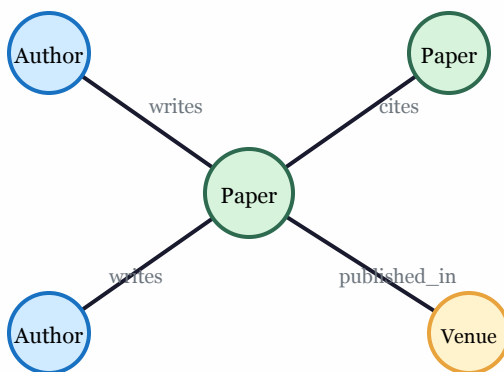
Questions about anything in this lesson? Ask your teacher — run `/teach` and ask about whatever is unclear. Next up: **Lesson 18 — Heterogeneous Graphs**, moving from theory back to a practical extension: graphs with multiple node and edge types.

Heterogeneous Graphs

Every architecture so far — GCN, GraphSAGE, GAT, GIN, graph transformers — has quietly assumed one node type and one edge type. A social network's "friend" edges, a molecule's bonds: one relation, one meaning, one shared weight matrix. Most real graphs aren't like that. A citation graph has **papers**, **authors**, and **venues** as node types, connected by **cites**, **writes**, and **published_in** edges. A financial graph has **accounts**, **merchants**, and **devices** linked by **transacts**, **owns**, and **logs_in_from** edges. These are **heterogeneous graphs**, and treating every edge the same way throws away exactly the information that makes them useful.

Why one weight matrix isn't enough

Recall the message-passing update from earlier lessons: a node aggregates transformed messages from its neighbors, typically via $\mathbf{h}_i' = \sigma(\sum_{j \in N(i)} \mathbf{W} \mathbf{h}_j)$, with a single learned $\mathbf{W}$ shared across every edge. Now consider the citation graph:



Three node types (Author, Paper, Venue) and three edge types (writes, cites, published_in). A "cites" edge and a "writes" edge connect completely different kinds of things for completely different reasons.

An author "writing" a paper and a paper "citing" another paper are not the same kind of relationship, even though both are just edges in the adjacency structure. Squeezing them through the same W forces the model to learn one transformation that has to work for two semantically unrelated operations. The fix seems obvious once stated: give each relation type its own transformation.

Key idea: A heterogeneous graph has a typed schema — a set of node types and a set of relation (edge) types, each of which may deserve its own weight matrix. This is a strict generalization of everything so far: a graph with one node type and one edge type is just the homogeneous special case.

R-GCN: relation-specific weight matrices

R-GCN (Relational GCN, Schlichtkrull et al. 2018) is the direct extension of GCN-style message passing to multiple relations. Instead of one shared W , every relation r gets its own W_r , and a node's update sums contributions per relation:

$$h_i' = \sigma \left(\sum_{r \in R} \sum_{j \in N_r(i)} (1/c_{i,r}) W_r h_j + W_0 h_i \right)$$

$N_r(i)$ is node i 's neighborhood *under relation r specifically*, $c_{i,r}$ is a normalization constant (analogous to the degree normalization from the GCN lesson, but computed per relation), and W_0 is a self-connection term carrying the node's own features forward — the relational analogue of the self-loop.

The parameter-blowup problem

If a knowledge graph has hundreds of relation types, R-GCN needs hundreds of full-sized weight matrices. That's both a huge parameter count and a rare-relation overfitting risk — a relation seen 20 times in training doesn't have enough data to learn a whole W_r from scratch. R-GCN's paper introduces two regularization schemes to solve this:

- **Basis decomposition:** every W_r is written as a linear combination of a small shared set of basis matrices $V_1 \dots V_B$, with only the per-relation coefficients a_{rb} being relation-specific: $W_r = \sum_{b=1}^B a_{rb} V_b$. Since $B \ll |R|$, this is effective weight-sharing across relations — structurally similar relations end up leaning on similar basis combinations.
- **Block-diagonal decomposition:** each W_r is constrained to be block-diagonal, so it can't mix features across blocks. This is a sparsity constraint rather than a sharing constraint — it encodes the assumption that latent features cluster into groups that interact strongly within a group and weakly across groups.

Analogy: Basis decomposition for R-GCN is the same move as depthwise-separable convolutions or low-rank weight factorization in standard deep learning — express a large family of transformations as combinations of a small shared basis, trading a bit of expressiveness for a large reduction in free parameters.

HAN: meta-paths and two levels of attention

HAN (Heterogeneous Graph Attention Network) takes a different angle. Rather than one weight matrix per raw edge type, HAN organizes computation around **meta-paths** — hand-specified composite relations like Author–Paper–Author ("co-authorship," two authors connected through a shared paper) or Author–Paper–Venue–Paper–Author ("published in the same venue"). For each meta-path, HAN builds a homogeneous neighborhood and applies GAT-style attention within it, then combines across meta-paths with a second attention layer:

- **Node-level attention** (intra-meta-path): for a fixed meta-path, weigh how much each meta-path neighbor matters — exactly the GAT mechanism from the architectures module, applied within one meta-path's neighborhood.

- **Semantic-level attention** (inter-meta-path): weigh how much each *meta-path itself* matters for the task — co-authorship might matter more than shared-venue for predicting research topic, for instance.

HAN's strength is interpretability: you can inspect which meta-paths and which neighbors the model leaned on. Its weakness is the meta-paths are a modeling choice a human has to make in advance — the model can't discover a useful composite relation you didn't think to define.

HGT: attention without hand-picked meta-paths

HGT (Heterogeneous Graph Transformer) removes that manual step. Instead of pre-defined meta-paths, HGT decomposes every edge into a **meta-relation triplet**: $\langle \text{source node type, edge type, target node type} \rangle$. Query, key, and value projections for the transformer-style attention (recall the graph transformer lesson) are parameterized *per triplet*, so a "Paper cites Paper" edge and an "Author writes Paper" edge each get their own attention parameterization automatically, without anyone hand-designing which composite paths matter. Stacking HGT layers lets the model discover higher-order composite relations implicitly, the way stacking any message-passing layer expands the receptive field.

Key idea: R-GCN, HAN, and HGT all answer the same question — "how do relation-specific transformations get parameterized without the parameter count exploding?" — with three different design choices: R-GCN shares parameters via a basis of matrices, HAN shares parameters by making a human pre-specify which composite relations (meta-paths) matter, and HGT shares parameters by attention over meta-relation triplets, learning composite structure automatically.

Model	Relation handling	Needs hand-picked meta-paths?
R-GCN	Per-relation weight matrix (basis / block-diagonal shared)	No — operates on raw edge types
HAN	GAT-style attention within each meta-path, then across meta-paths	Yes — a human defines the meta-paths
HGT	Transformer attention parameterized per $\langle \text{type}, \text{edge}, \text{type} \rangle$ triplet	No — composite relations emerge from stacked layers

Check your understanding

1. In R-GCN, why does each relation type need its own weight matrix W_r ?

Because different relations transform neighbor features differently

Because every relation needs a unique bias

Because shared weights increase total training time

Because each node type needs more features

A single shared matrix forces semantically unrelated relations (e.g., "cites" and "writes") to be squeezed through the same transformation. Relation-specific W_r lets the model learn that these relations mean structurally different things.

2. What problem does R-GCN's basis decomposition address?

Too many parameters across many relations

Too few labeled training graph examples

Too slow message passing computation speed

Too little attention across graph layers

With hundreds of relation types, storing a full W_r per relation is both expensive and prone to overfitting on rare relations. Basis decomposition writes every W_r as a combination of a small shared set of basis matrices, cutting the free parameter count sharply.

3. What is a key practical advantage of HGT over HAN?

It does not require hand-designed meta-paths chosen manually

It does not require any labeled node features

It does not use self-attention over neighborhoods ever

It does not scale to large graphs well

HAN needs a human to pre-specify which meta-paths (e.g., Author–Paper–Author) are worth attending over. HGT instead parameterizes attention per $\langle \text{source type, edge type, target type} \rangle$ triplet, so composite relational patterns emerge from stacking layers rather than manual specification.

Primary source

Watch or read: Stanford CS224W Lecture 9 (<https://cs224w.stanford.edu/>) on heterogeneous graphs, and read Schlichtkrull et al., "Modeling Relational Data with Graph Convolutional Networks" (<https://arxiv.org/abs/1703.06103>) (2018) for the R-GCN

architecture, including the basis and block-diagonal decompositions in full mathematical detail.

Questions about anything in this lesson? Ask your teacher — run `/teach` and ask about whatever is unclear. Next up: **Lesson 19 — Knowledge Graphs and Embedding**, where relation types become first-class citizens of the embedding space itself, not just a parameterization choice inside message passing.

← Lesson 17: Graph Transformers

Lesson 19: Knowledge Graphs and Embedding →

Knowledge Graphs and Embedding

A **knowledge graph** is a heterogeneous graph (Lesson 18) viewed through a specific lens: every edge is a fact, written as a **triple** (h, r, t) — head entity, relation, tail entity. "(Obama, born_in, Hawaii)." "(Aspirin, treats, Headache)." The graph itself is the database. The task that dominates this space isn't node classification — it's **knowledge graph completion**: given millions of known facts, predict plausible missing ones. Does the triple (Obama, born_in, Kenya) belong in the graph? Almost certainly not. Does (Aspirin, treats, Fever)? Probably.

This lesson is deliberately not about message passing. It's about a different, older idea — recall **DeepWalk** / **node2vec** from the shallow-embeddings lesson, where you learn a lookup-table vector per node with no neural network aggregation at all. **Knowledge graph embedding (KGE)** methods do the same thing for multi-relational graphs: learn a vector per entity *and* a vector (or matrix) per relation, such that some simple scoring function over (h, r, t) is high for true triples and low for corrupted ones.

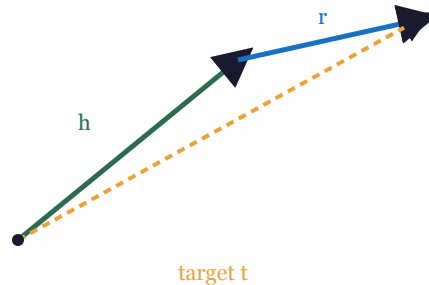
Key idea: KGE methods don't aggregate neighbor messages the way GCN/GAT/GraphSAGE do. They learn shallow embeddings directly, optimized so a scoring function $f(h, r, t)$ separates true triples from corrupted ones. The relation itself becomes a learned object in the embedding space — not just a parameterization choice for a weight matrix, as it was for R-GCN in Lesson 18.

TransE: relations as translations

TransE (Bordes et al., 2013) has arguably the simplest idea in the whole GNN/graph-representation-learning space: if (h, r, t) is true, the tail's embedding should be reachable from the head's embedding by *adding the relation vector*:

$$h + r \approx t$$

$$\text{score}(h, r, t) = -\|h + r - t\|$$



TransE's geometric picture: starting at h , walking along the relation vector r should land close to t . The score is (negative) distance between $h+r$ and the actual t .

TransE is trained with a margin-based ranking loss: true triples should score higher than corrupted ones (formed by swapping in a random head or tail) by at least some margin. It's cheap, scales to huge knowledge graphs, and works well for one-to-one relations. But the translation picture breaks down for relational patterns that are common in real knowledge graphs:

- **Symmetric relations** (e.g., `is_married_to`): if $h + r \approx t$ and $t + r \approx h$ both hold, adding the two gives $2r \approx 0$, forcing r toward the zero vector — which then can't distinguish this relation from any other near-zero relation.
- **One-to-many / many-to-one relations** (e.g., `born_in`, where many people share a birth country): TransE wants $h + r$ to land at one specific point, but many valid tails would need to sit at that same point, collapsing entities that should stay distinct.

DistMult: a bilinear score

DistMult replaces translation with a bilinear form. Each relation is a *diagonal matrix* (equivalently, a vector r , since only the diagonal is used), and the score is:

$$\text{score}(h, r, t) = h^T \text{diag}(r) t = \sum_i h_i r_i t_i$$

This is cheap (elementwise multiply-then-sum, no full matrix) and empirically strong. But look closely at the formula: swapping h and t leaves the score completely unchanged, since elementwise multiplication commutes. That means DistMult is fundamentally **symmetric** — it cannot represent antisymmetric relations like `parent_of`, where $(A, \text{parent_of}, B)$ being true should make $(B, \text{parent_of}, A)$ false. DistMult would score both triples identically.

Complex: moving to complex-valued embeddings

Complex (Trouillon et al., 2016) fixes exactly this gap by embedding entities and relations as *complex* vectors instead of real ones, and scoring with the real part of a Hermitian (conjugated) inner product:

$$\text{score}(h, r, t) = \text{Re}(\sum_i h_i r_i \text{conj}(t_i))$$

The conjugate on t is what breaks the symmetry that trapped DistMult: swapping h and t no longer leaves the score unchanged, because conjugation is applied to whichever embedding sits in the tail position. This lets a single, still-simple bilinear-style model represent both symmetric and antisymmetric relations, which is why Complex (and its later relatives) generally outperforms both TransE and DistMult on standard knowledge graph completion benchmarks.

Analogy: Going from DistMult to Complex is the same trick as going from a real-valued to a complex-valued rotation matrix in signal processing — the complex domain gives you a natural notion of "direction" (phase) that a purely real, symmetric bilinear form can't express.

Model	Scoring intuition	Weak spot
TransE	$h + r \approx t$ (translation)	Symmetric & many-to-many relations
DistMult	$h^T \text{diag}(r) t$ (bilinear, real)	Antisymmetric relations
ComplEx	$\text{Re}(h^T \text{diag}(r) \text{conj}(t))$ (bilinear, complex)	More parameters, still no explicit composition

Where this reconnects to message passing

These three methods are all "shallow" — no neighborhood aggregation, just a lookup-table embedding per entity and per relation. In practice, KGE and GNN ideas combine: R-GCN from Lesson 18 is often used as an *encoder* that produces entity embeddings via relational message passing, which then feed into a DistMult-style *decoder* for the actual link-prediction score. The GNN handles structural context; the shallow scoring function handles the final triple-plausibility judgment. This encoder/decoder split is worth remembering — it reappears whenever a graph task needs both "understand the neighborhood" and "score a specific relationship."

Check your understanding

1. Why does TransE struggle to represent a symmetric relation like "is_married_to"?

It forces the relation vector to zero

It forces the entity vectors to align

It forces the embedding dimension to shrink

It forces the loss function to diverge

If $h + r \approx t$ and $t + r \approx h$ both need to hold, adding them gives $2r \approx 0$, pushing the relation vector toward zero and making it indistinguishable from other near-zero relations.

2. What's the essential change that lets ComplEx model antisymmetric relations, unlike DistMult?

ComplEx uses complex-valued embeddings with a conjugate operation

ComplEx uses larger embeddings with more training data

ComplEx uses graph convolutions with additional attention layers

ComplEx uses random negative sampling with harder examples

DistMult's bilinear score is symmetric in h and t , so it can never separate a true triple from its reversed form. ComplEx embeds in complex space and conjugates the tail embedding, which breaks that symmetry and lets the model score $(A, \text{parent_of}, B)$ differently from $(B, \text{parent_of}, A)$.

3. In knowledge graph completion, what does the trained scoring function let us do?

Rank candidate tail entities for a triple

Rank candidate node degrees for a graph

Rank candidate edge counts for a batch

Rank candidate loss values for a model

Given a partial triple $(h, r, ?)$, the scoring function is evaluated against every candidate entity as the tail, and the candidates are ranked by score. High-ranked, unseen triples are the model's proposed missing facts.

Primary source

Read: Hamilton, "Graph Representation Learning" (https://www.cs.mcgill.ca/~wlh/grl_book/), **Chapter 4** (relational data and knowledge graph embeddings), plus Stanford CS224W Lecture 10 (<https://cs224w.stanford.edu/>). For the primary sources on the three methods themselves: Bordes et al., "Translating Embeddings for Modeling Multi-relational Data" (<https://papers.nips.cc/paper/5071-translating-embeddings-for-modeling-multi-relational-data>) (TransE, NeurIPS 2013); Yang et al., "Embedding Entities and Relations for Learning and Inference in Knowledge Bases" (<https://arxiv.org/abs/1412.6575>) (DistMult, 2015); Trouillon et al., "Complex Embeddings for Simple Link Prediction" (<https://arxiv.org/abs/1606.06357>) (ComplEx, 2016).

Questions about anything in this lesson? Ask your teacher — run `/teach` and ask about whatever is unclear. Next up: **Lesson 20 — Dynamic and Temporal GNNs**, where the graph itself stops being a static object and starts changing over time — the setting that matters most for your AML/fraud interest.

Dynamic and Temporal GNNs

Every architecture in this course so far has quietly made one more assumption on top of "one node type, one edge type": the graph doesn't change. Compute the adjacency structure once, run message passing, get an embedding. Real graphs you'd actually care about for fraud and AML aren't like that. An account isn't fixed in place with a static set of counterparties — it accumulates a stream of transactions, each one a new, timestamped edge. The graph you'd want to reason about at 3pm is not the graph you'd want to reason about at 9am. This lesson is about GNNs that treat time as a first-class input rather than an afterthought.

Two fundamentally different ways to have "time"

Before any architecture, there's a modeling fork that matters more than which specific model you pick:

- **Discrete-time (snapshot) graphs:** the graph is a sequence of static graphs $G_1, G_2, \dots, G_T$ — e.g., "the transaction graph as of the end of each day." Time is a coarse index. Two events one minute apart and one 23 hours apart, if they land in the same snapshot, become indistinguishable.
- **Continuous-time (event-stream) graphs:** the graph is a stream of individually timestamped events, arriving at irregular real-valued times, with no binning at all. This is what a transaction feed actually looks like.

Key idea: Financial transaction graphs are continuous-time event streams, not snapshots — money moves whenever it moves, not on a schedule the graph waits for. Forcing them into fixed snapshots throws away exactly the fine-grained timing signal (burst detection, time-since-last-transfer, velocity) that fraud and layering patterns often hinge on. This fork is developed in full in the Temporal GNN Patterns reference doc ([../reference/temporal-gnn-patterns.html](https://reference/temporal-gnn-patterns.html)) — treat

this lesson as the walkthrough and that doc as the lookup table you'll come back to.

EvolveGCN: evolve the weights, not the nodes

EvolveGCN (Pareja et al., 2020) stays in the discrete-time, snapshot world, but makes an unusual choice about what "learns over time." Instead of a static GCN applied identically to every snapshot, the GCN's *weight matrices themselves* are evolved snapshot-to-snapshot by an RNN:

- **EvolveGCN-H** treats the weight matrix $W^{(t)}$ as the hidden state of a GRU, updated using the current snapshot's node embeddings as input.
- **EvolveGCN-O** treats $W^{(t)}$ as the output of an RNN, which is then fed forward as the input to produce $W^{(t+1)}$.

Either way, at each snapshot you get a fresh GCN — same architecture, different weights — run on that snapshot's structure. No per-node memory persists; everything about a node's representation is recomputed from the current graph and the current (evolved) weights. This matters for graphs where the node set itself changes a lot: EvolveGCN doesn't need to have seen a given node before to produce weights for it, unlike methods that maintain per-node state directly.

TGAT: time-aware attention over raw history

TGAT (Xu et al., 2020) moves to continuous time. Its central device is a **functional time encoding**: a learned mapping from a time delta (how long ago did this neighbor's edge form, relative to now) into a feature vector, grounded in Bochner's theorem from harmonic analysis — conceptually similar to the sinusoidal positional encodings you've seen in sequence Transformers, but learned and continuous rather than fixed and discrete. This time-encoding is concatenated onto node features, and a self-attention layer — structurally the same idea as the graph

transformer lesson — attends over a node's chronologically-ordered neighborhood, weighted partly by how temporally relevant each past neighbor is.

Crucially, TGAT keeps *no persistent state*. Every time you want an embedding for a node, you recompute it by re-attending over its recent raw neighbor history. That's simple and stateless, but it means the "context window" of history you attend over has to be re-scanned every single query.

TGN: give every node a memory

TGN (Rossi et al., 2020) keeps TGAT's time-encoded attention idea but adds the piece TGAT is missing: a persistent **memory vector** per node, updated incrementally as events happen rather than recomputed from scratch each time. Three pieces work together:

1. **Message function:** when an event touches a node (e.g., a transaction), compute a message from the event features and the two involved nodes' current memory.
2. **Memory updater** (typically a GRU): fold that message into the node's memory, producing an updated memory vector.
3. **Graph embedding module:** when a prediction is actually needed, read the current memory vectors of the node and its neighbors (using TGAT-style temporal attention) to produce the final embedding.

The memory is what makes this different from TGAT in practice: a node's entire relevant history is compressed into a fixed-size vector that updates incrementally, the same way an RNN's hidden state summarizes a sequence without re-reading it. TGN's own ablation results show TGAT is effectively a special case of the TGN framework with the memory module switched off.

Analogy: EvolveGCN is like periodically retraining a static model's weights as new batches of data arrive. TGAT is like a Transformer that re-reads its whole recent context window on every query. TGN is like an RNN with an explicit

per-entity hidden state — it never needs to re-read history because it's already carrying a running summary of it forward.

DyRep, briefly

DyRep (Trivedi et al., 2019) predates TGN and shares its state-based instinct, but frames the problem as a **temporal point process** with two coupled dynamics operating at different time scales: *topological evolution* (new edges/nodes appearing) and *node interaction dynamics* (activity between nodes that already exist). An attention-based network produces the representations that drive both processes. It's worth knowing by name mainly because TGN's memory mechanism is a direct intellectual descendant of DyRep's per-node state updates — DyRep got there first, with a more explicit point-process framing.

Check your understanding

1. Which architecture evolves the GNN's own weight matrices over time via an RNN, rather than maintaining per-node state?

EvolveGCN, which regenerates weights per graph snapshot

TGAT, which regenerates weights per graph snapshot

TGN, which regenerates weights per graph snapshot

DyRep, which regenerates weights per graph snapshot

EvolveGCN runs an otherwise-ordinary GCN on each snapshot, but its weight matrix is produced fresh at every snapshot by an RNN (GRU or LSTM) that has seen the weights from prior snapshots. TGAT, TGN, and DyRep are all continuous-time models that don't work in snapshots at all.

2. What does TGN's memory module store per node, and how does it update?

A per-node vector updated by a message function

A per-edge vector updated by a message function

A per-node matrix updated by a message function

A per-graph vector updated by a message function

TGN maintains one memory vector per node. Each event touching that node produces a message (via the message function), which a GRU-based memory updater folds into that node's memory vector incrementally — no full history rescan needed.

3. What is the key practical reason continuous-time models matter more than snapshot models for transaction graphs?

Transactions arrive continuously, not in fixed time windows

Transactions arrive continuously, not in labeled fraud categories

Transactions arrive continuously, not in encrypted account numbers

Transactions arrive continuously, not in duplicated node features

Transaction feeds are inherently irregular, real-time event streams. Binning them into fixed snapshots discards exactly the fine-grained timing signal (bursts, velocity, time-since-last-transfer) that fraud and layering detection often depends on most.

Reference doc

For the full side-by-side breakdown of how EvolveGCN, TGAT, TGN, and DyRep each handle time, plus a decision framework for which to reach for, see the Temporal GNN Patterns ([../reference/temporal-gnn-patterns.html](https://reference/temporal-gnn-patterns.html)) reference doc built alongside this lesson.

Primary source

Read: Rossi et al., "Temporal Graph Networks for Deep Learning on Dynamic Graphs" (<https://arxiv.org/abs/2006.10637>) (2020), and Pareja et al., "EvolveGCN: Evolving Graph Convolutional Networks for Dynamic Graphs" (<https://arxiv.org/abs/1902.10191>) (2020). For the time-encoding mechanism specifically, see Xu et al., "Inductive Representation Learning on Temporal Graphs" (<https://arxiv.org/abs/2002.07962>) (TGAT, 2020).

Questions about anything in this lesson? Ask your teacher — run `/teach` and ask about whatever is unclear. Next up: **Lesson 21 — GNNs for Fraud and AML Detection**, where these temporal patterns meet the domain they were largely built for.

← Lesson 19: Knowledge Graphs and Embedding

Lesson 21: GNNs for Fraud and AML Detection →

GNNs for Fraud and AML Detection

This is the lesson the last three have been building toward. Heterogeneous graphs (Lesson 18) gave you typed nodes and relations — accounts, merchants, devices; owns, transacts_with, logs_in_from. Knowledge graph embedding (Lesson 19) gave you the idea of scoring relationships directly. Temporal GNNs (Lesson 20) gave you the machinery for graphs that arrive as continuous event streams. Financial crime detection needs all three at once: a heterogeneous, continuously-evolving graph, scored under conditions no standard GNN benchmark prepares you for.

What makes fraud graphs different

Most of the architectures in this course were designed and benchmarked on citation networks, molecules, and social graphs — settings where classes are reasonably balanced, the graph is mostly static, and there's no adversary rewriting the data to evade the model. Fraud and AML graphs violate all three assumptions:

- **Extreme class imbalance.** Confirmed fraud or money-laundering activity is typically a tiny fraction of a percent of all transactions. A model that predicts "legitimate" for everything gets near-perfect accuracy and is completely useless. This forces specific handling — resampling, class-weighted or focal losses, or fraud-aware neighbor sampling — rather than the standard cross-entropy setup used elsewhere in this course.
- **Adversarial, adaptive behavior.** A molecule doesn't change its bonds to evade a toxicity classifier. A fraud ring does change its behavior once a detection pattern starts working against it. The data distribution isn't stationary the way it is in most GNN benchmarks — it actively shifts in response to the model, which is why one-time training on historical data quietly decays in production.
- **Camouflage via legitimate-looking patterns.** Fraudulent accounts deliberately transact with legitimate ones to blend in — a pattern sometimes called *relation camouflage*. This produces **heterophily**: a fraud node's

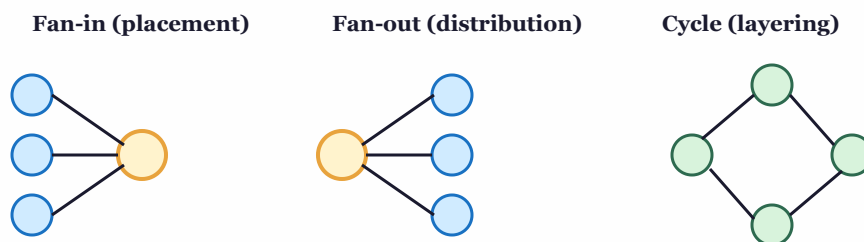
immediate neighbors are disproportionately legitimate, which is the opposite of the assumption (similar nodes connect to similar nodes) that architectures like GCN implicitly lean on. It's a big part of why plain message-passing GNNs, applied naively, can underperform on fraud detection.

- **Low-latency requirements.** A molecular property predictor can take seconds; a payment-fraud model often needs to score a transaction before it settles — sub-second, in production. This rules out any approach that requires reprocessing the entire graph's history for every new event.
- **Structure itself is signal.** Individual transaction features (amount, currency, merchant category) are only part of the picture. The *shape* of the surrounding subgraph is often the strongest signal a layering scheme leaves behind.

Key idea: Fraud graphs aren't just "graphs with a harder label." Class imbalance, adversarial drift, heterophilic camouflage, and latency constraints each independently break an assumption that the standard GNN toolkit (Modules 1–5) was built around. Applying GNNs to AML well means knowing which assumption you're relaxing and why.

Structure as a first-class signal: fan-in, fan-out, cycles

Classic money-laundering typologies leave a topological fingerprint, independent of any single transaction's features:



Fan-in (many small "smurfed" deposits converging on one account — placement), fan-out (one account rapidly distributing funds outward), and cycles (funds routed back toward their origin through

intermediaries — classic layering). None of these are visible from any single transaction's features; they only exist at the subgraph level.

This is exactly the kind of pattern message passing over multiple hops is suited to surface — a 1-layer GNN sees only direct neighbors, but the receptive-field argument from the message-passing module means a deeper (or attention-based, longer-range) model can implicitly detect a several-hop cycle or an unusually dense fan-in, the way it detects any other multi-hop structural motif.

Mapping Lesson 20's patterns onto a detection pipeline

A realistic AML pipeline combines pieces from the last three lessons rather than picking just one:

- **Graph schema:** heterogeneous, as in Lesson 18 — accounts, merchants, devices, IP addresses as distinct node types, each transaction/ownership/login as a distinct typed edge. An R-GCN- or HGT-style encoder respects that a "shared device" edge means something structurally different from a "wire transfer" edge.
- **Time framing:** continuous-time, not snapshot — transactions post whenever they post. This is precisely the argument from Lesson 20 for reaching toward TGN-style architectures: a persistent per-account memory vector, updated incrementally as each new transaction arrives via a message function, avoids re-scanning an account's entire history every time a new transaction needs a real-time risk score.
- **Latency:** TGN's incremental memory update is what makes near-real-time scoring tractable at all — recomputing a full-history embedding (TGAT-style) for every single incoming transaction doesn't scale the same way when transactions arrive at high volume and a decision is needed before settlement.
- **Class imbalance:** handled at the training-data level (targeted negative/positive sampling around known fraud subgraphs, since fraud tends to cluster rather than scatter uniformly) and at the loss level (class-weighted or focal loss so the rare positive class isn't drowned out).

- **Camouflage / heterophily:** some fraud-specific GNN variants deliberately relax or reweight the neighbor-aggregation step (rather than trusting that "neighbor = similar"), specifically because fraud graphs violate the homophily assumption baked into architectures like plain GCN.
- **Explainability:** AML decisions frequently feed into regulatory filings (suspicious activity reports), which need a human-readable justification, not just a score. Attention weights from GAT/HAN/HGT-style layers, or the specific neighbor events a TGN's memory update leaned on, are a natural source of that justification — a materially different requirement than most benchmark GNN tasks ever impose.

Analogy: Think of the pipeline as three questions stacked on top of each other, one per lesson: "what kinds of things and relationships exist here?" (Lesson 18's typed schema), "how do I score a specific relationship or fact?" (Lesson 19's scoring-function idea, echoed in edge-level fraud scores), and "how do I keep that score current as new events keep arriving?" (Lesson 20's continuous-time memory). Fraud detection needs an answer to all three simultaneously, which is why it's a late lesson in this course rather than an early one.

Check your understanding

1. What makes class imbalance especially severe in fraud/AML data compared to typical benchmark graph datasets?

Fraudulent transactions represent a tiny minority overall

Fraudulent transactions represent a large majority overall

Fraudulent transactions represent a random sample overall

Fraudulent transactions represent a balanced portion overall

Confirmed fraud or laundering activity is typically a tiny fraction of a percent of all transactions, so a model that predicts "legitimate" everywhere scores near-perfect accuracy while being useless — this forces resampling, reweighted losses, or fraud-aware sampling rather than standard cross-entropy training.

2. Why is graph structure itself treated as a signal in AML detection, beyond node and edge features?

Layering schemes create distinctive multi-hop transaction structural patterns

Layering schemes create distinctive multi-hop transaction feature vectors

Layering schemes create distinctive single-hop transaction structural patterns

Layering schemes create distinctive multi-hop merchant loyalty programs

Fan-in, fan-out, and cycle patterns (placement, distribution, layering) only exist at the subgraph level, across several hops — no single transaction's features reveal them. This is exactly the kind of multi-hop pattern deeper or attention-based message passing is suited to surface.

3. Why do continuous-time models like TGN suit AML transaction monitoring better than periodic batch retraining pipelines?

They can score new transactions before funds settle

They can score new transactions after funds settle

They can score old transactions before funds settle

They can score new merchants before funds settle

TGN's incremental per-node memory update lets a new transaction be scored right away, without rescanning an account's full history, which is what makes sub-second risk scoring before settlement tractable — a snapshot or periodic-retraining pipeline can't offer that.

Primary source

Read: Cheng, Zou, Xiang & Jiang, "Graph Neural Networks for Financial Fraud Detection: A Review" (<https://arxiv.org/abs/2411.05815>) (2024, published in *Frontiers of Computer Science*). It reviews over 100 studies and gives a unified framework for categorizing GNN approaches to financial fraud detection, including the class-imbalance and camouflage/heterophily issues covered above.

Questions about anything in this lesson? Ask your teacher — run `/teach` and ask about whatever is unclear. This closes Module 6 (Lessons 18–21) — the specialized, real-world-graph topics your AML/fraud background makes especially relevant. Next up: **Lesson 22 — Deep Generative Models for Graphs**, an optional-module shift toward generation rather than prediction.

← Lesson 20: Dynamic and Temporal
GNNs

Lesson 22: Deep Generative Models for Graphs
→

Deep Generative Models for Graphs

Every lesson so far has asked a GNN to *consume* a graph and produce a prediction. This lesson flips the question: can a model *produce* a graph? Given a dataset of molecules, can we train something that generates new, plausible molecules? This is a genuinely different problem from message passing, and it's worth seeing why — briefly, since this module is optional broader literacy rather than core material.

Why graph generation is harder than image or text generation

A VAE for images outputs a fixed-size pixel grid; an autoregressive language model outputs one token at a time in a fixed left-to-right order. Graphs have neither property:

- **No canonical ordering.** The same graph can be written down as $|V|!$ different adjacency matrices, one per node permutation (recall permutation invariance from Lesson 1). A generative model that scores a specific matrix has to somehow not penalize a correct graph just because it came out in a different node order.
- **Discrete, structured output.** Nodes and edges are discrete objects with hard constraints (an edge either exists or doesn't), unlike a continuous pixel intensity.

Two influential approaches handle this permutation problem in opposite ways: GraphVAE generates the whole graph at once and pays a matching cost after the fact; GraphRNN generates the graph piece by piece and avoids most of the matching cost by imposing an ordering during training.

GraphVAE: generate all at once, then match

GraphVAE (Simonovsky & Komodakis, 2018) follows the standard VAE recipe you already know: an encoder maps a graph to a latent vector, a decoder maps the latent vector back to a graph, and the model is trained to reconstruct its inputs through that bottleneck. The twist is entirely in what the decoder outputs and how reconstruction error is measured.

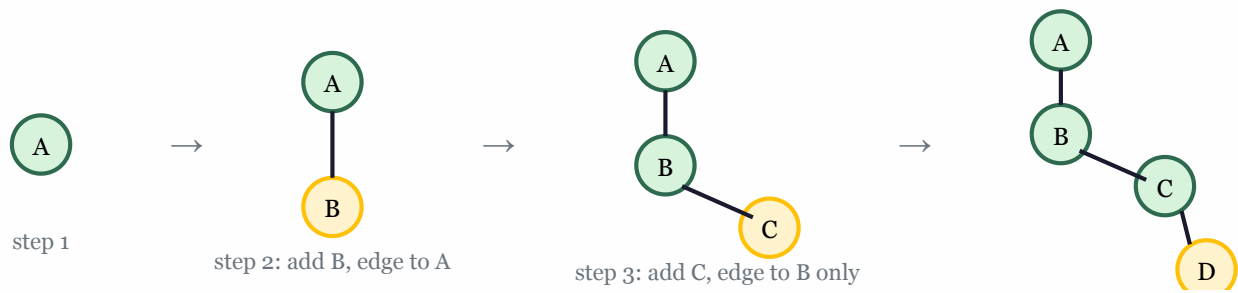
The decoder outputs a **probabilistic, fully-connected graph of a fixed maximum size** in one shot: an existence probability for every possible node, a predicted class for every node, and an existence probability plus edge type for every possible node pair — essentially a soft adjacency matrix and a soft feature matrix, produced directly by an MLP, with no message passing involved in decoding.

Key idea: Because the decoder has no idea which of the $|V|!$ orderings of the target graph to aim for, GraphVAE can't just compare its output matrix to the target matrix entry-by-entry. It first solves an approximate **graph-matching** problem — searching for the permutation of the output graph's nodes that best aligns it to the target graph — and only then computes reconstruction loss against that best-aligned version.

That matching step is expensive: the graph-matching subroutine GraphVAE uses is an approximate relaxation, but still costs on the order of $O(n^4)$ for an n -node graph, and the $O(n^2)$ all-pairs output itself doesn't scale. In practice, GraphVAE is demonstrated on small graphs — tens of nodes, not thousands. It's an elegant proof of concept for how to handle permutation invariance in a generative loss, more than a production-scale generator.

GraphRNN: generate one piece at a time

GraphRNN (You, Ying, Ren, Hamilton & Leskovec, 2018) sidesteps the all-pairs output entirely by treating graph generation as **sequence generation**: add one node, decide its edges to existing nodes, add the next node, repeat, until a stop token is emitted.



GraphRNN generation: at each step, the graph-level RNN adds a node, then the edge-level RNN decides — one Bernoulli draw at a time — whether an edge exists to each recent node, before the next node is added.

Two RNNs work together: a **graph-level RNN** tracks the state of the graph built so far and initializes the state for each new node; a **edge-level RNN** then steps through the existing nodes and predicts, one Bernoulli (edge / no-edge) decision at a time, whether the new node connects to each of them. Training uses teacher forcing against real graphs with a binary cross-entropy loss, and generation at inference time samples from those Bernoulli outputs until an end-of-sequence signal is produced.

Key idea: The remaining problem is that a graph still has no single canonical node order to train the RNN on. GraphRNN's fix is to always linearize training graphs with a **breadth-first search (BFS) ordering**. This collapses the astronomically many equivalent orderings down to a much smaller set of BFS-consistent ones, and — because BFS visits nodes in shells — it also guarantees a new node can only connect to a bounded window of *recent* nodes, not the entire graph seen so far. That's what keeps each edge-level decision cheap and lets GraphRNN scale to graphs far larger than GraphVAE can handle.

GraphVAE vs. GraphRNN, side by side

	GraphVAE	GraphRNN
Generation style	One-shot (whole adjacency matrix)	Sequential (autoregressive, node by node)
Handles permutation via	Graph matching at loss time	Fixed BFS ordering during training
Practical scale	Small graphs (tens of nodes)	Much larger graphs

Both papers predate the current wave of diffusion- and flow-based graph generators, which is where most active generative-graph research has since moved — but GraphVAE and GraphRNN are still the cleanest illustration of the core problem: *any* graph generator has to decide how it will cope with the fact that a graph has no canonical ordering, and every later method inherits that same choice in one form or another.

Check your understanding

1. Why does GraphVAE need a graph-matching step before computing its reconstruction loss?

To reduce total parameter count

To align arbitrary node orderings

To increase generated graph diversity

To speed up decoder training

The decoder's output matrix and the target graph's matrix may represent the same graph under two different node orderings. Graph matching searches for the permutation that best aligns them so the reconstruction loss compares like with like, rather than penalizing a correct graph for an arbitrary relabeling.

2. GraphRNN orders nodes via breadth-first search before training.

What does this primarily achieve?

It caps the number of prior nodes considered

It removes the need for an edge-level RNN

It converts the graph into a dense matrix

It guarantees every newly generated graph is connected

BFS ordering collapses the many equivalent node orderings into a smaller, consistent set, and because BFS visits nodes in shells, a new node's edge decisions only need to look back at a bounded recent window rather than every node generated so far.

3. A researcher needs to generate graphs with thousands of nodes and wants training to stay tractable. Which approach fits better, and why?

GraphRNN, since generation and loss scale near-linearly

GraphVAE, since matching cost stays constant always

GraphRNN, since it avoids using neural networks

GraphVAE, since dense matrices compress large graphs

GraphVAE's $O(n^2)$ all-pairs output and roughly $O(n^4)$ matching cost make it impractical past small graphs. GraphRNN's sequential generation with a bounded BFS window avoids both bottlenecks, which is why it was demonstrated on graphs an order of magnitude larger.

Primary source

Read: William L. Hamilton, "Graph Representation Learning"

(https://www.cs.mcgill.ca/~wlh/grl_book/), **Chapters 8–9**, for the systematic treatment of generative graph models. For the original papers: Simonovsky & Komodakis, "GraphVAE: Towards Generation of Small Graphs Using Variational Autoencoders" (2018) (<https://arxiv.org/abs/1802.03480>) and You et al., "GraphRNN: Generating Realistic Graphs with Deep Auto-regressive Models" (2018) (<https://arxiv.org/abs/1802.08773>). For lecture-style coverage, see Stanford CS224W's "Deep Generative Models for Graphs" (<http://snap.stanford.edu/class/cs224w-2024/slides/16-deep-generation.pdf>) slides.

Questions about anything in this lesson? Ask your teacher — run `/teach` and ask about whatever is unclear. Next up: **Lesson 23 — LLMs + GNNs**, the final lesson of the course, on how language models and graph models are being combined.

← Lesson 21: GNNs for Fraud and AML Detection

Lesson 23: LLMs + GNNs →

LLMs + GNNs

This is a fast-moving area with no single foundational paper the way GCN or GAT have one — what follows is a synthesis across several distinct research threads, current as of writing, rather than one canonical result. Treat this lesson as a map of vocabulary and framings for reading further, not a settled textbook chapter.

The pairing makes sense once you name the asymmetry: LLMs are extremely good at open-ended reasoning over text but have no native notion of graph structure; GNNs are good at structure but only consume whatever features you hand them. Most current work combines them along one of a few recurring patterns.

Pattern 1: serialize the graph as text, let the LLM reason directly

The simplest combination skips GNNs entirely: turn the graph into a text prompt (a node list, an edge list, or some structured description) and ask an LLM to answer questions about it — "is there a path from A to D?", "what's node X's degree?" — using its general reasoning ability.

Google Research's "Talk like a Graph" (<https://arxiv.org/abs/2310.04560>) (Fatemi, Halcrow & Perozzi, ICLR 2024) studied this directly and found that **how** you encode the graph as text matters enormously: switching encoding schemes swung LLM accuracy on graph reasoning tasks by anywhere from 4.8 to 61.8 percentage points on the same underlying task, and performance degraded further as graphs grew larger or tasks required deeper multi-hop structural reasoning.

Key idea: An LLM reasoning over a serialized graph has no architectural permutation invariance or message-passing inductive bias — recall from Lesson 1 that this was the whole reason GNNs exist. It's approximating structural reasoning through general-purpose text understanding, which

works surprisingly well on small graphs and simple queries but is not a substitute for a GNN on tasks that need reliable multi-hop structure.

Pattern 2: LLMs feed better features into a GNN

A large share of real graphs are **text-attributed**: citation networks where nodes are papers with abstracts, product graphs where nodes are listings with descriptions, social graphs where nodes have bios. Traditionally these get embedded with something like TF-IDF or a small pretrained encoder to become the node feature matrix X from Lesson 2. A recent line of work instead uses an LLM to generate or enrich that text first — for instance, prompting an LLM to produce an explanation or expanded description for a node before embedding it — and only then hands the resulting features to an ordinary GNN for message passing.

A recent survey (Li et al., "A Survey of Graph Meets Large Language Model," IJCAI 2024 (<https://arxiv.org/abs/2311.12399>)) organizes this and related work into a three-way taxonomy worth knowing as vocabulary:

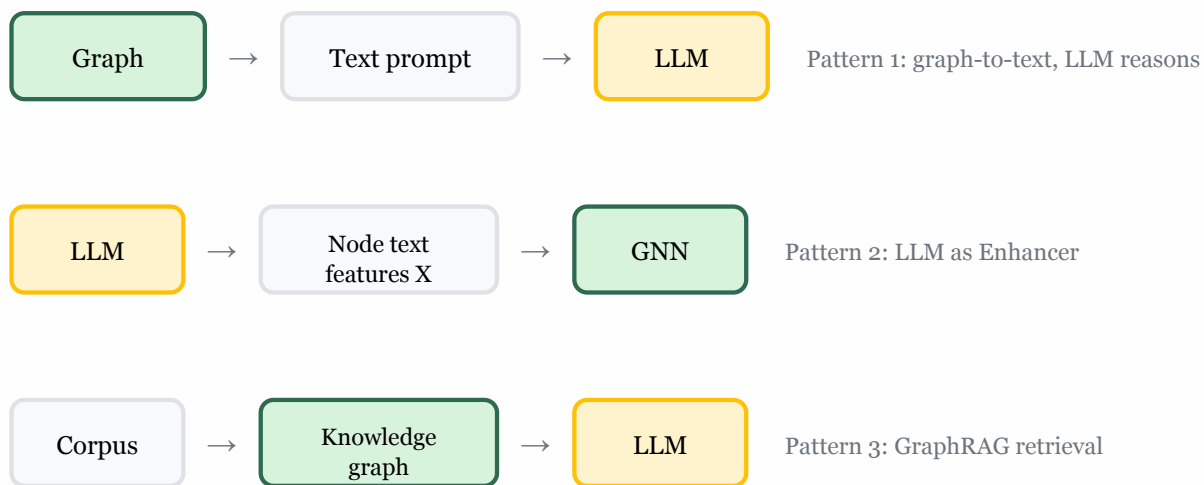
- **LLM as Enhancer** — the LLM improves node/edge features or representations that a GNN then consumes (the pattern just described).
- **LLM as Predictor** — the LLM itself makes the final prediction, given some serialization or summary of the graph (Pattern 1, pushed further).
- **GNN–LLM Alignment** — the two are trained jointly, with a shared or aligned representation space so structural and textual signals reinforce each other rather than one simply feeding the other.

Pattern 3: graphs as retrieval structure for grounded generation (GraphRAG)

Standard retrieval-augmented generation (RAG) retrieves text chunks by embedding similarity. **GraphRAG** (Microsoft Research) instead builds a knowledge graph from the source corpus first — extracting entities and

relationships, then clustering the graph into communities and generating a summary for each community — and uses that graph structure to drive retrieval: local queries traverse nearby entities and relationships, while broad "what are the themes in this corpus" queries pull from the higher-level community summaries. This targets a known weakness of plain embedding-similarity RAG: answering questions that require connecting information scattered across many documents, which a flat similarity search tends to miss.

Analogy: Plain RAG is like retrieving individual index cards that mention your query terms. GraphRAG is like retrieving index cards *plus* the folder structure connecting them, so the model can follow "these three entities co-occur across five documents" the way a GNN follows edges — except here the "graph" was extracted and summarized by an LLM in the first place, and the traversal is done for retrieval rather than for a differentiable forward pass.



Three recurring integration patterns. The arrow direction tells you who's providing structure and who's providing reasoning.

What this doesn't replace

None of these patterns make Lessons 4–21 obsolete. Message passing is still the tool of choice when you need reliable, differentiable, permutation-equivariant

structural computation over a graph you can actually put in a tensor — a transaction network for AML detection, a molecule, a citation graph you're training end-to-end. LLM integration is most useful at the edges: producing richer input features, answering open-ended questions about a graph in natural language, or grounding generation in a corpus's structure. The two are complementary tools, not competitors, and knowing when a plain GNN is the right, cheaper, more reliable answer is itself part of the literacy this module is trying to give you.

Check your understanding

1. In the "LLM as Enhancer" pattern, what role does the LLM play in a GNN pipeline?

It generates richer text-based node features

It replaces message passing with attention

It pools node embeddings into graphs

It samples negative edges for training

The LLM enriches or generates the text associated with nodes (e.g., in a citation or product graph), which is then embedded into the node feature matrix X . The GNN still does the structural work of message passing on top of those improved features.

2. According to "Talk like a Graph," what most affects an LLM's accuracy on graph reasoning tasks?

The chosen text encoding of the graph

The chosen programming language for the prompts

The chosen color scheme for the diagrams

The chosen font size in the prompts

The paper found accuracy swings of 4.8 to 61.8 percentage points purely from changing how the same graph was serialized into text, alongside effects from graph size and task type — the encoding choice is not a cosmetic detail.

3. What does GraphRAG add on top of standard vector-similarity RAG retrieval?

Graph-structured retrieval over extracted entity relationships

Image-based retrieval over extracted pixel patterns

Audio-based retrieval over extracted speech signals

Random retrieval over shuffled document chunks

GraphRAG builds a knowledge graph from the corpus, clusters it into communities, and summarizes each community, then retrieves along that graph structure instead of (or alongside) flat embedding similarity — helping answer questions whose answer is scattered across many documents.

Primary source

Watch/read: Stanford CS224W's "GNNs & LLMs"

(<https://web.stanford.edu/class/cs224w/slides/18-LLMs+GNN.pdf>) lecture slides. For the specific claims in this lesson: Fatemi, Halcrow & Perozzi, "Talk like a Graph: Encoding Graphs for Large Language Models" (<https://arxiv.org/abs/2310.04560>) (ICLR 2024); Li et al., "A Survey of Graph Meets Large Language Model: Progress and Future Directions" (<https://arxiv.org/abs/2311.12399>) (IJCAI 2024); and Microsoft Research's GraphRAG blog

post (<https://www.microsoft.com/en-us/research/blog/graphrag-unlocking-llm-discovery-on-narrative-private-data/>), with code at the [microsoft/graphrag](https://github.com/microsoft/graphrag) repository.

Questions about anything in this lesson? Ask your teacher — run `/teach` and ask about whatever is unclear.

Course complete. Twenty-three lessons ago, Lesson 1 opened with "what if the data has neither grid nor sequence structure, but relational structure?" You can now trace a message-passing computation by hand; explain why permutation equivariance is the design constraint that makes GNNs work at all; compare GCN, GAT, GraphSAGE, and GIN by inductive bias, attention, and expressive power; place a new architecture on the WL-expressiveness hierarchy; and reason about heterogeneous, knowledge, and temporal graphs — including the AML/fraud-detection pipeline from Lesson 21. That's everything MISSION.md set out as success: you should be able to read a GNN paper, follow its architecture and training procedure, and judge when a GNN (versus an LLM, versus a shallow embedding) is the right tool.

What's genuinely next, in rough order of value: **(1)** if AML/fraud is where you'll apply this, go deeper than Lesson 21's survey level — follow the temporal-GNN papers (TGN, EvolveGCN) in RESOURCES.md into the fraud-detection literature that cites them. **(2)** Join a community from RESOURCES.md's Wisdom section (r/MachineLearning, the PyG discussion forum, Papers With Code) to keep pace with a field that moves as fast as this lesson's topic did. **(3)** Pick one paper — GCN, GAT, GraphSAGE, TGN, or something AML-specific — and read it start to finish without help. That's the real test of whether this course did its job.